NECMETTİN ERBAKAN ÜNİVERSİTESİ
Mühendislik ve Mimarlık Fakültesi
Bilgisayar Mühendisliği Bölümü

PROJE TASARIM RAPORU

Akıllı Oto Galeri Sistemi

(AOGS)

Web Mimarisi (React + Spring Boot + PostgreSQL)

Ders:	BIL204 Yazılım Mühendisliği
Bölüm / Şube:	Bilgisayar Mühendisliği 2. Sınıf
Grup Adı:	AOGS
Proje Adı:	AOGS
Teslim Tarihi:	04.05.2026

GRUP ÜYELERİ

Ad Soyad	Öğrenci No	Görev / Sorumlu Modül
Emre Kuzal	24100011003	Proje Yöneticisi & Satış / Raporlama Modülleri (Backend: Satis Entity, SatisRepository, SatisService, SatisController, RaporlayiciService, RaporController, DTO'lar)
Zeki Erman	25100011462	Araç Modülü (Backend: Arac/Otomobil/SUV/Ticari/Motosiklet Entity'leri, AracRepository, AracService, AracController) — Frontend: Dashboard, AracListesi, AracEkle, Raporlama UI
Orhan Akman	24100011013	Müşteri Modülü (Backend: Musteri Entity, MusteriRepository, MusteriService, MusteriController) — Frontend: MusteriYonetimi, SatisIslemi, SigortaMuayene UI; Layout (Sidebar & TopBar)
Osman Berat Paşa	24100011048	Ekspertiz Modülü (Backend: Ekspertiz Entity, HasarSeviyesi enum, EkspertizRepository, EkspertizService, EkspertizController)
Halil İbrahim Dinç	23100011047	Bildirim & Sigorta/Muayene Modülleri (Backend: Calisan, SigortaBilgisi, MuayeneBilgisi Entity'leri; CalisanRepository, SigortaRepository, MuayeneRepository; BildirimService, BildirimController; GlobalExceptionHandler)
Eray Gök	24100011027	Entegrasyon & Test (Frontend: api/client.js axios istemcisi, theme/Tailwind yapılandırması; Backend: CorsConfig, GaleriApplication; JUnit testleri ve dokümantasyon)

*Bu belge; Akıllı Oto Galeri Sistemi (AOGS) projesinin yazılım tasarımını modern bir Web Mimarisi (Client-Server) yaklaşımıyla detaylı biçimde sunmaktadır. Analiz aşamasında hazırlanan SRS belgesinde sistemin **ne yapacağı** tanımlanmıştı; bu raporda ise sistemin bu gereksinimleri **nasıl gerçekleştireceği** akademik üslupla belgelendirilmektedir. Belge IEEE yazılım mühendisliği standartlarına uygun olarak hazırlanmıştır.*

İÇİNDEKİLER

1. GİRİŞ 5
- 1.1. Sistemin Amacı 5
- 1.2. Tasarım Hedefleri 6
  - 1.2.1. Kullanılabilirlik 6
  - 1.2.2. Genişletilebilirlik 7
  - 1.2.3. Bakım Yapılabilirlik 7
  - 1.2.4. Performans 8
  - 1.2.5. Güvenilirlik 8
  - 1.2.6. Taşınabilirlik 9
  - 1.2.7. Görsel Tutarlılık 9
- 1.3. Tasarım Ödünleşimleri (Trade-off'lar) 10

2. ÜST DÜZEY YAZILIM MİMARİSİ 12
- 2.1. Alt Sistem Ayrıştırması 12
  - 2.1.1. Sunum Katmanı (React) 14
  - 2.1.2. Controller Katmanı (REST API) 15
  - 2.1.3. Service Katmanı (İş Mantığı) 16
  - 2.1.4. Repository Katmanı (Veri Erişimi) 17
- 2.2. Donanım / Yazılım Eşlemesi 18
- 2.3. Kalıcı Veritabanı Yönetimi 20
- 2.4. Erişim Kontrolü ve Güvenlik 21
- 2.5. Küresel Kontrol Akışı 22
- 2.6. Sınır Koşulları 24
  - 2.6.1. Başlatma (Initialization) 24
  - 2.6.2. Sonlandırma (Termination) 25
  - 2.6.3. Hata Durumları (Failure) 25

3. ALT DÜZEY TASARIM 27
- 3.1. Nesne Tasarım Kararları 27
- 3.2. Son Nesne Tasarımı (UML Sınıf Diyagramı) 29
- 3.3. Veritabanı Şeması (ER Diyagramı) 31
- 3.4. Paketler / Modül Yapısı 33
  - 3.4.1. Backend Paketleri 33
  - 3.4.2. Frontend Modülleri 34
- 3.5. Sınıf Arayüzleri 36
  - 3.5.1. Model Katmanı (JPA Entity'leri) 36
  - 3.5.2. Repository Katmanı (Spring Data JPA) 44
  - 3.5.3. Service Katmanı (İş Mantığı) 47
  - 3.5.4. Controller Katmanı (REST API) 51
  - 3.5.5. Sunum Katmanı (React Bileşenleri) 53
- 3.6. Tasarım Desenleri 55
  - 3.6.1. Dependency Injection 55
  - 3.6.2. Repository Pattern 57
  - 3.6.3. DTO (Data Transfer Object) Pattern 58
  - 3.6.4. Strategy Pattern (JPA Specification) 59

SÖZLÜK (GLOSSARY) 61

REFERANSLAR 63

ŞEKİLLER LİSTESİ

Şekil 2.1: Alt Sistem Ayrıştırma Diyagramı (Client-Server Mimarisi)
Şekil 2.2: Konuşlanma (Deployment) Diyagramı
Şekil 2.3: Sınır Koşulları Durum Diyagramı
Şekil 2.4: Küresel Kontrol Akışı (REST API Tabanlı)
Şekil 3.1: Paket Diyagramı (Backend + Frontend)
Şekil 3.2: UML Sınıf Diyagramı (JPA Entity'leri ve Katmanlar)
Şekil 3.3: Varlık-İlişki (ER) Diyagramı
Şekil 3.4: Kullanım Durumu (Use Case) Diyagramı
Şekil 3.5: UI Navigasyon Akış Diyagramı
Şekil 3.6: Satış Modülü Aktivite Diyagramı
Şekil 3.7: Dependency Injection Tasarım Deseni
Şekil 3.8: Repository Tasarım Deseni
Şekil 3.9: DTO Tasarım Deseni
Şekil 3.10: Strategy Tasarım Deseni (JPA Specification)

TABLOLAR LİSTESİ

Tablo 1.1: Tailwind CSS Üzerinden AOGS Renk Paleti
Tablo 1.2: Tasarım Ödünleşimleri Özeti
Tablo 2.1: Sunucu Donanım Gereksinimleri
Tablo 2.2: Yazılım Bağımlılıkları (Backend / Frontend)
Tablo 2.3: REST API İşlev Yetki Matrisi
Tablo 3.1: Sınıf İlişkileri ve Tasarım Gerekçeleri
Tablo 3.2: Veritabanı İndeks Stratejisi
Tablo 3.3: Backend Paket Yapısı ve Sorumluluklar
Tablo 3.4: Frontend Modül Yapısı
Tablo 3.5: Tasarım Desenleri Özeti

1. GİRİŞ

Bu belge, Akıllı Oto Galeri Sistemi (AOGS) projesinin yazılım tasarımını IEEE yazılım mühendisliği standartlarına uygun biçimde detaylı olarak sunmaktadır. Analiz aşamasında hazırlanan Yazılım Gereksinimleri Spesifikasyonu (SRS) belgesinde sistemin **ne yapacağı** tanımlanmıştı; bu tasarım raporunda ise sistemin bu gereksinimleri **nasıl gerçekleştireceği** ayrıntılı olarak açıklanmaktadır. Belge; üst düzey mimari kararlardan veritabanı şemasına ve sınıf bazlı tasarıma kadar tüm tasarım kararlarını akademik üslupla belgelendirmeyi amaçlamaktadır.

1.1. Sistemin Amacı

Akıllı Oto Galeri Sistemi (AOGS), oto galeri işletmelerinde geleneksel olarak kâğıt tabanlı ya da dağınık elektronik tablolarla yürütülen operasyonel süreçleri; modern bir **web tabanlı uygulama** üzerinde tek bir merkezi platformda toplamayı hedefleyen bir yazılım sistemidir. Sistemin temel amacı; araç stok yönetimi, müşteri ilişkileri, satış işlemleri, dijital ekspertiz raporlaması, sigorta ve muayene tarihi takibi ile raporlama gibi iş süreçlerini dijitalleştirerek galeri çalışanlarının iş yükünü azaltmak ve işletme verimliliğini artırmaktır.

Sistem; istemci (client) tarafında **React** tabanlı modern bir tek sayfa uygulaması (SPA), sunucu (server) tarafında ise **Java Spring Boot** ile geliştirilmiş bir REST API ve **PostgreSQL** veritabanından oluşmaktadır. İstemci ile sunucu arasındaki iletişim; HTTP protokolü üzerinden JSON formatlı veri alışverişi ile gerçekleştirilmektedir. Bu modern Client-Server mimarisi; eşzamanlı çoklu kullanıcı erişimini, kalıcı veri saklamayı ve farklı cihazlarda (PC, tablet, mobil) tutarlı kullanıcı deneyimini desteklemektedir.

Hedef kullanıcı kitlesi; küçük ve orta ölçekli oto galeri işletmelerinin çalışanlarıdır. Kullanıcıların ileri düzey bilgisayar becerisine sahip olması beklenmemekte; dolayısıyla sistem, sezgisel ve mobil uyumlu (responsive) bir arayüz ile tasarlanmıştır. Sekiz temel ekran (Dashboard, Araç Listesi, Araç Ekleme, Müşteri Yönetimi, Satış İşlemi, Ekspertiz, Sigorta/Muayene Takibi, Raporlama) üzerinden kullanıcılar tüm iş süreçlerini en fazla üç tıklama ile yönetebilmektedir.

Proje, Necmettin Erbakan Üniversitesi Bilgisayar Mühendisliği Bölümü **BIL204 Yazılım Mühendisliği** dersi kapsamında, altı kişilik bir geliştirici ekibi tarafından geliştirilmektedir. Backend Java 17 ve Spring Boot 3.2 ile, frontend ise React 18, Vite ve Tailwind CSS ile kodlanmaktadır. Veri saklama için **PostgreSQL** kullanılmakta olup; üretim ortamında bulut tabanlı yönetilen bir veritabanı hizmeti (Supabase) tercih edilmektedir.

> **BİLGİ**
>
> Bu sürüm, projenin önceki masaüstü tabanlı tasarımının modern bir Web Mimarisine taşınmış halidir. **Java Spring Boot** backend ile **React** frontend, RESTful bir API üzerinden iletişim kurar. Bellek-içi veri tutma yerine kalıcı bir **PostgreSQL** veritabanı kullanılmaktadır.

1.2. Tasarım Hedefleri

Bu bölümde AOGS projesinin işlevsel olmayan (non-functional) gereksinimleri ve kalite hedefleri detaylandırılmaktadır. Her tasarım hedefi; neden önemli olduğu, sistemde nasıl sağlanacağı ve ölçülebilir kriterleri ile birlikte sunulmaktadır.

1.2.1. Kullanılabilirlik (Usability)

AOGS'un temel kullanıcı kitlesi, teknik olmayan galeri çalışanlarıdır. Dolayısıyla arayüzün sezgisel, tutarlı ve öğrenmesi kolay olması en öncelikli tasarım hedeflerindendir. Bu hedef aşağıdaki tasarım kararları ile desteklenmektedir:

- **Mobil Uyumlu (Responsive) Tasarım:** Frontend, **Tailwind CSS**'in utility-first yaklaşımı ile geliştirildiğinden tüm ekranlar otomatik olarak PC, tablet ve mobil cihazlara uyum sağlar. Aynı kod tabanı ile farklı ekran genişliklerinde tutarlı bir kullanıcı deneyimi elde edilmektedir.
- **Üç Tıklama Kuralı:** Herhangi bir iş akışı, ana ekrandan (Dashboard) başlayarak en fazla üç tıklama ile tamamlanabilmektedir.
- **Sabit Sidebar Navigasyonu:** Sekiz ana ekrana erişim, sol taraftaki sabit navigasyon menüsünden sağlanmaktadır. Kullanıcı hangi ekranda olduğunu her zaman görsel olarak takip edebilmektedir (aktif menü vurgusu).
- **Anlık Geri Bildirim:** Her kullanıcı eylemi (kayıt, silme, satış, hata) toast bildirimleri ve modal pencereler ile anında bildirilmektedir. REST API'den dönen yanıtlar, React state yönetimi ile arayüzde milisaniyeler içinde yansıtılır.
- **Doğrulama ve Hata Mesajları:** Form alanlarındaki hatalı girişler (negatif fiyat, geçersiz TC kimlik, boş zorunlu alanlar) hem istemci tarafında React doğrulamasıyla, hem de sunucu tarafında Spring Validation (JSR-380) ile çift kademeli olarak denetlenir.

1.2.2. Genişletilebilirlik (Extensibility)

AOGS, gelecekte yeni özelliklerin mevcut koda minimum müdahale ile eklenebileceği biçimde tasarlanmıştır. Bu hedef, hem nesne yönelimli programlama ilkeleri hem de katmanlı web mimarisi sayesinde sağlanmaktadır:

- **Soyut Sınıflar (Abstraction):** `Arac` soyut JPA Entity'si ve `Kisi` `@MappedSuperclass` sayesinde yeni araç türleri (örn: Elektrikli Araç) ya da yeni kişi türleri mevcut kodun değiştirilmesine gerek kalmadan sisteme eklenebilmektedir.
- **RESTful API:** Backend'in tüm işlevleri, standart HTTP metotları (GET, POST, PUT, DELETE) üzerinden yayınlanır. İlerleyen aşamalarda mobil uygulama, üçüncü taraf entegrasyonlar veya başka servisler aynı API'yi yeniden kullanabilir.
- **Modüler Paket Yapısı:** `com.galeri.model`, `com.galeri.repository`, `com.galeri.service`, `com.galeri.controller` şeklinde net biçimde ayrılmış paketler sayesinde her modül bağımsız olarak geliştirilebilmektedir.
- **Bileşen Bazlı Frontend:** React'in bileşen mimarisi sayesinde yeni sayfalar ve UI öğeleri mevcut kodu etkilemeden eklenebilir. Tailwind utility class'ları ile yeni bileşenler mevcut tasarım sistemine doğal olarak entegre olur.
- **JPA Specification API:** Dinamik sorgu üretimi için Strategy deseni uygulanmıştır; yeni filtreleme kriterleri mevcut Repository'leri değiştirmeden eklenebilir.

1.2.3. Bakım Yapılabilirlik (Maintainability)

Altı kişilik bir ekibin paralel olarak çalıştığı bu projede, kodun okunabilir, bakımı kolay ve tutarlı olması kritik önem taşımaktadır. Aşağıdaki tasarım kararları bu hedefe hizmet etmektedir:

- **SOLID İlkeleri:** Tek Sorumluluk, Açık-Kapalı, Liskov Yerine Geçme, Arayüz Ayırma ve Bağımlılık Ters Çevirme ilkeleri tasarımın temelini oluşturmaktadır. Spring'in Dependency Injection mekanizması, son ilkeyi framework seviyesinde kolaylaştırır.
- **Türkçe Javadoc Yorumları:** Tüm public metotlar ve önemli sınıflar Türkçe Javadoc formatında belgelendirilmiştir.
- **Katmanlı Mimari:** Controller → Service → Repository → Entity katmanlarının net ayrımı, bir katmandaki değişikliğin diğerlerini etkilemesini önler.
- **Anlamlı İsimlendirme:** Sınıf, metot ve değişken adları İngilizce olmakla birlikte, proje gereği Türkçe domain terminolojisini (`aracEkle`, `musteriAra`) yansıtmaktadır.
- **Git Flow Stratejisi:** `main` (kararlı sürüm), `develop` (geliştirme) ve `feature/<modul-adi>` dalları ile organize bir versiyon kontrolü uygulanmaktadır.

1.2.4. Performans (Performance)

Sistemin performansı; istemci tarafında React'in verimli sanal DOM (virtual DOM) algoritması, sunucu tarafında ise **veritabanı indeksleme** ve **SQL sorgu optimizasyonu** teknikleri ile sağlanır. Tüm temel işlemlerin, normal yük altında 200 milisaniyenin altında tamamlanması hedeflenmektedir.

- **B-Tree İndeksleme:** Sık sorgulanan sütunlar (plaka_no, tc_kimlik, satis_tarihi, marka, durum) üzerinde PostgreSQL B-Tree indeksleri oluşturulmuştur. Bu sayede arama operasyonları O(log n) karmaşıklığında gerçekleşir; milyonlarca satırlı tablolarda dahi milisaniyelik yanıt süreleri elde edilir.
- **Tekil Kısıtlamalar (Unique Constraints):** Plaka numarası ve TC kimlik gibi benzersiz alanlar üzerinde unique B-Tree indeksleri tanımlanmıştır. Hem tekrarlı kayıtların önüne geçilir hem de hızlı arama sağlanır.
- **SQL Agregasyon:** Raporlama metrikleri (toplam ciro, kâr, en çok satılan marka), `SUM`, `AVG`, `COUNT`, `GROUP BY` SQL operatörleri ile veritabanı tarafında tek geçişte hesaplanır. Veriler uygulama sunucusuna gereksiz yere taşınmaz.
- **JPA Lazy Loading:** İlişkili varlıklar (`@ManyToOne(fetch = FetchType.LAZY)`) yalnızca ihtiyaç duyulduğunda yüklenir; gereksiz JOIN sorguları önlenir.
- **İstemci Tarafı Optimizasyonu:** React 18'in eşzamanlı render mekanizması ve Vite'ın hızlı geliştirme sunucusu (HMR) sayesinde UI tepki süreleri minimize edilir.

1.2.5. Güvenilirlik (Reliability)

Sistem; geçersiz veri girişleri, ağ kesintileri ve beklenmeyen durumlarda çökmeden çalışmaya devam edebilmelidir:

- **Çift Katmanlı Doğrulama:** Form alanları hem istemci tarafında (React + JS) hem de sunucu tarafında (`@Valid` + Bean Validation) doğrulanır. Sunucu, istemciye güvenmez.

- **İşlem (Transaction) Yönetimi:** Service metotları `@Transactional` ile sarmalanır. Birden fazla veritabanı işlemi atomik biçimde yürütülür; herhangi bir adımda hata oluşursa tüm değişiklikler geri alınır (ROLLBACK).
- **Global Exception Handler:** `@RestControllerAdvice` ile tüm controller'larda yakalanmamış istisnalar merkezi bir noktada işlenir; kullanıcıya tutarlı hata yanıtları döndürülür.
- **Referans Bütünlüğü:** Foreign key kısıtlamaları sayesinde tutarsız veri kaydı veritabanı seviyesinde engellenir.

1.2.6. Taşınabilirlik (Portability)

Web tabanlı mimari sayesinde sistem büyük ölçüde platform bağımsızdır. İstemci tarafında modern bir tarayıcı yeterlidir; herhangi bir kurulum gerekmez. Sunucu tarafında uygulama, Java Virtual Machine üzerinde çalıştığı için Linux, Windows veya macOS işletim sistemlerinde aynı şekilde çalışabilir. Veritabanı; Supabase, AWS RDS veya yerel bir PostgreSQL kurulumu olabilir; uygulama konfigürasyon dosyasında basit bir bağlantı dizgisi değişikliği yeterlidir.

1.2.7. Görsel Tutarlılık (Visual Consistency)

AOGS arayüzü; **turuncu-beyaz** tema etrafında kurgulanmış, tutarlı bir görsel dil sunmaktadır. Renkler, boşluklar, yazı tipleri ve bileşen stilleri **Tailwind CSS** utility class'ları üzerinden tanımlanmıştır. Bu sayede:

- Renkler tek bir merkezi yapılandırma dosyasında (`tailwind.config.js`) `aogs.*` ad alanı altında tanımlanır.
- Tüm bileşenlerde aynı utility class'lar (`bg-aogs-birincil`, `text-aogs-metin`, `rounded-xl` vb.) kullanılır; stil tekrarı önlenir.
- Tema değişikliği için yalnızca yapılandırma dosyasının güncellenmesi yeterlidir; tüm UI otomatik olarak yeni renklere uyar.

Renk Rolü	Tailwind Sınıfı / Hex	Kullanım Alanı
Birincil (Ana Turuncu)	`aogs-birincil / #FF6B35`	Ana vurgu, aktif menü, başlıklar, birincil butonlar
İkincil (Açık Turuncu)	`aogs-ikincil / #F7931E`	Hover durumu, ikincil vurgular
Arka Plan	`aogs-arkaplan / #FFFBF7`	Genel arka plan, içerik alanı
Kart Arka Planı	`aogs-kart / #FFFFFF`	Kart kontenerleri, form alanları, modal pencereler
Kenar (Soft Turuncu)	`aogs-kenar / #FFE5D0`	Border'lar, ayırıcı çizgiler
Metin (Koyu)	`aogs-metin / #2C3E50`	Standart metin rengi
Sidebar (Koyu)	`aogs-sidebar / #2C3E50`	Sol navigasyon menüsü arka planı
Başarı (Yeşil)	`aogs-basari / #27AE60`	Başarılı işlem mesajları, badge'ler
Uyarı (Kırmızı)	`aogs-uyari / #E74C3C`	Hata mesajları, silme onayları

Tablo 1.1: *Tailwind CSS Üzerinden AOGS Renk Paleti*

1.3. Tasarım Ödünleşimleri (Trade-off'lar)

Her yazılım tasarımı, birbiriyle çelişen kalite hedefleri arasında bilinçli tercihler yapmayı gerektirir. AOGS tasarımında aşağıdaki ödünleşimler ele alınmış ve projeye uygun kararlar verilmiştir.

1.3.1. Mimari Karmaşıklığı vs. Ölçeklenebilirlik

Tek bir uygulamada hem UI hem iş mantığı hem de veri saklama mantığını barındıran monolitik mimariler, küçük projelerde geliştirme sürecini hızlandırır. Ancak AOGS için Client-Server mimarisinin tercih edilmesinin nedeni; ileride çoklu cihaz desteği (mobil uygulama, tablet kioskları), eşzamanlı çoklu kullanıcı erişimi ve mikroservis tabanlı evrime kapı aralamaktır. İlk geliştirme süresi bir miktar uzasa da elde edilen mimari esneklik bu maliyeti fazlasıyla karşılamaktadır.

1.3.2. Performans vs. Tutarlılık

Yüksek hızlı yanıt için bazı sorgular önbelleğe alınabilir; ancak bu durum verinin anlık tutarlılığını riske atar. AOGS, finansal hassasiyeti olan bir sistem olduğundan tutarlılık her zaman ön planda tutulmuştur. Tüm yazma işlemleri `@Transactional` ile sarmalanır ve veritabanından doğrudan okunur; potansiyel önbelleğe alma ileri sürümler için planlanmıştır.

1.3.3. Bağlılık (Dependency) Sayısı vs. Üretkenlik

Projede Spring Boot, Spring Data JPA, Hibernate, React, Vite, Tailwind, axios, Recharts gibi birçok üçüncü taraf kütüphane kullanılmaktadır. Daha az bağımlılık ile minimum kütüphane kullanmak; yapay yazılım yığınlarını sade tutar ancak yeniden icat maliyeti getirir. AOGS, endüstri standardı olgun kütüphaneleri tercih ederek geliştirme süresini ve hata olasılığını minimize eden tarafı seçmiştir.

1.3.4. Sunucu Maliyeti vs. Çoklu Erişim

Masaüstü tek-kullanıcı yaklaşımı, sunucu maliyeti getirmez; ancak veriler tek bir cihazda kalır, paylaşım ve eşzamanlılık imkânsızdır. AOGS'un seçtiği web tabanlı yaklaşım; sürekli çalışan bir backend sunucusu ve veritabanı bulundurmayı gerektirir. Buna karşılık galeri sahibi, çalışanları ve depodaki personel aynı anda farklı cihazlardan sisteme erişebilir, veriler kalıcıdır ve yedeklenebilir.

Ödünleşim	Tercih Edilen Taraf	Gerekçe
Mimari Karmaşıklığı vs. Ölçeklenebilirlik	Ölçeklenebilirlik	Client-Server, çoklu kullanıcı ve gelecekteki mobil/entegrasyon ihtiyaçlarını destekler.
Performans vs. Tutarlılık	Tutarlılık	Finansal işlemlerde anlık doğru veri kritiktir; @Transactional ile garanti edilir.
Bağımlılık Sayısı vs. Üretkenlik	Üretkenlik	Spring, JPA, React, Tailwind gibi olgun kütüphaneler proje süresini kısaltır.
Sunucu Maliyeti vs. Çoklu Erişim	Çoklu Erişim	Galeri ortamında birden fazla çalışan eşzamanlı çalışır; kalıcılık zorunludur.

Tablo 1.2: *AOGS Tasarım Ödünleşimleri Özeti*

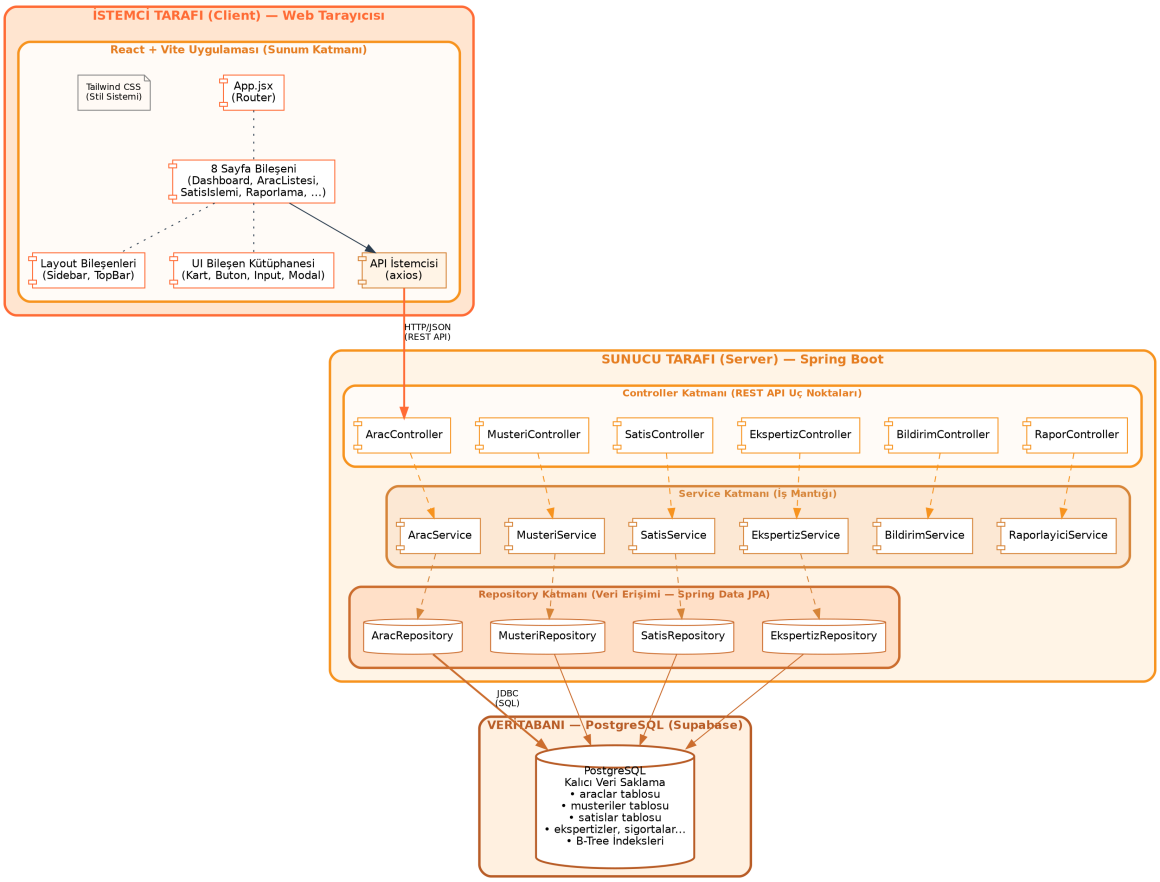
2. ÜST DÜZEY YAZILIM MİMARİSİ

Bu bölümde AOGS'un üst düzey yazılım mimarisi; alt sistem ayrıştırması, donanım/yazılım eşlemesi, kalıcı veri yönetimi, erişim kontrolü, küresel kontrol akışı ve sınır koşulları başlıkları altında detaylı olarak ele alınmaktadır. AOGS, modern bir **Client-Server Mimarisi** üzerine inşa edilmiştir. Frontend (istemci) ve backend (sunucu) ayrı süreçler olarak çalışır; aralarındaki tüm iletişim **RESTful API** üzerinden HTTP/JSON protokolleriyle gerçekleştirilir. Bu yapı; modülerlik, ölçeklenebilirlik, çoklu cihaz desteği ve farklı istemci türleri (web, mobil) için tek bir backend kullanımını mümkün kılar.

2.1. Alt Sistem Ayrıştırması

AOGS'un alt sistem ayrıştırması, **ayrım endişeleri ilkesi** (Separation of Concerns) ve **Client-Server Mimarisi** ilkeleri gözetilerek yapılmıştır. Sistem üç ana parçadan oluşur: **İstemci (Client)**, **Sunucu (Server)** ve **Veritabanı**. Sunucu kendi içinde dört katmana ayrılır: Controller (REST API), Service (İş Mantığı), Repository (Veri Erişimi) ve Model (JPA Entity'ler).

Üst katmanlar yalnızca alt katmanlara bağımlıdır. İstemci ile sunucu arasındaki tek temas noktası REST API'dir; sunucunun iç yapısı (Service, Repository, Entity) istemciden tamamen gizlenmiştir. Benzer şekilde sunucu, veritabanına yalnızca Repository katmanı üzerinden erişir; SQL dağılımı tüm uygulamada engellenir.



Şekil 2.1: *AOGS Alt Sistem Ayrıştırma Diyagramı — Client-Server mimarisinin katmanlı gösterimi.*

> **BİLGİ**
>
> Katmanlar arası iletişim **tek yönlüdür**: İstemci → Controller → Service → Repository → Veritabanı. Ters yönlü bağımlılıklar (örneğin Repository'nin Controller'ı çağırması) tasarım gereği yasaklanmıştır. Sunucu yanıtları, ters sırada katmanlar üzerinden geri döner.

2.1.1. Sunum Katmanı (React Frontend)

Sunum katmanı, kullanıcı ile sistem arasındaki tek iletişim arayüzüdür. **React 18**, **Vite** derleme aracı ve **Tailwind CSS** kullanılarak geliştirilen bu Tek Sayfa Uygulaması (Single Page Application — SPA), istemcinin tarayıcısında çalışır. Sunum katmanı hiçbir iş mantığı içermez; yalnızca *kullanıcıdan girdi alma, REST API'ye iletme, dönen yanıtı görselleştirme* görevini üstlenir.

Sunum katmanı şu bileşenlerden oluşur:

- **Sayfalar (Pages):** Sekiz ana ekran — Dashboard, Araç Listesi, Araç Ekleme, Müşteri Yönetimi, Satış İşlemi, Ekspertiz, Sigorta/Muayene, Raporlama. Her sayfa bağımsız bir React bileşenidir.
- **Layout Bileşenleri:** Sidebar (sol navigasyon menüsü) ve TopBar (üst başlık çubuğu) tüm sayfalarda ortaktır.
- **UI Kütüphanesi:** Yeniden kullanılabilir bileşenler — Kart, Buton, Input, Modal, Tablo — tek bir merkezi modülde tanımlanmıştır. Tüm bileşenler Tailwind utility class'ları ile stillendirilir.
- **API İstemcisi:** `axios` tabanlı bir istemci (`api/client.js`) tüm REST çağrılarını merkezi olarak yönetir. Her endpoint için tip-güvenli fonksiyonlar sunar (`aracApi.liste()`, `satisApi.yap(...)` vb.).
- **Yönlendirme (Routing):** `React Router` ile sayfa geçişleri sayfa yenilenmesi olmadan, sadece URL değişerek gerçekleşir. Bu sayede SPA deneyimi elde edilir.

Frontend; geliştirme sırasında Vite'ın hızlı yenileme (HMR) özelliğini kullanır. Üretim için `npm run build` komutu ile statik JavaScript, CSS ve HTML dosyaları üretilir; bu dosyalar herhangi bir web sunucusunda (Nginx, Apache, CDN) servis edilebilir.

2.1.2. Controller Katmanı (REST API)

Controller katmanı, istemciden gelen HTTP isteklerini karşılayan ilk katmandır. Spring Boot'un `@RestController` anotasyonu ile işaretlenmiş sınıflar; HTTP metotlarını (`@GetMapping`, `@PostMapping`, `@PutMapping`, `@DeleteMapping`) URL yollarına eşler ve gelen JSON gövdelerini Java nesnelerine (`@RequestBody`) çevirir.

AOGS sisteminde altı Controller bulunur:

- **AracController:** `/api/araclar` — Araç CRUD operasyonları, arama ve filtreleme.
- **MusteriController:** `/api/musteriler` — Müşteri yönetimi, TC ile arama.
- **SatisController:** `/api/satislar` — Satış işlemleri, iptal, fatura.
- **EkspertizController:** `/api/ekspertizler` — Ekspertiz raporları.
- **BildirimController:** `/api/bildirimler` — Sigorta ve muayene takibi.

- **RaporController:** `/api/raporlar` — Dashboard özeti, aylık/marka/ödeme şekli analizleri.

Controller katmanı iş mantığı içermez; gelen veriyi doğrular, uygun Service metodunu çağırır ve dönen sonucu HTTP yanıtına çevirir. Hata durumlarında `@RestControllerAdvice` ile tanımlanan Global Exception Handler devreye girer ve standart hata yanıtları üretir.

2.1.3. Service Katmanı (İş Mantığı)

Service katmanı, AOGS'un kalbi niteliğindedir. Tüm iş kuralları, veri doğrulamaları ve karmaşık iş akışı koordinasyonu bu katmanda yer alır. Service sınıfları `@Service` anotasyonu ile işaretlenir; Spring IoC Container otomatik olarak bu sınıfları singleton bean'ler olarak yönetir ve gerekli bağımlılıkları (constructor injection) enjekte eder.

AOGS'un altı Service sınıfı şunlardır:

- **AracService:** Araç ekleme, silme, güncelleme, arama; stok değeri hesaplama.
- **MusteriService:** Müşteri kaydı, TC ile arama, müşteri-satış eşleştirmesi.
- **SatisService:** Satış gerçekleştirme (tek atomik işlem ile araç durumu güncelleme), satış iptal, kâr hesaplama, fatura üretimi.
- **EkspertizService:** Ekspertiz rapor oluşturma, araç bazlı ekspertiz geçmişi sorgulama.
- **BildirimService:** Sigorta/muayene takibi, yaklaşan tarihler için uyarı üretme.
- **RaporlayiciService:** Gelir-gider raporu, en çok satılan marka, çalışan performansı ve diğer iş zekâsı raporları.

Service metotları, birden fazla Repository çağrısı yapan kritik iş akışlarında `@Transactional` anotasyonu ile sarmalanır. Bu sayede ya tüm değişiklikler veritabanına kaydedilir ya da hata durumunda tümü geri alınır (atomicity).

2.1.4. Repository Katmanı (Veri Erişim Katmanı)

Repository katmanı, sunucu ile veritabanı arasındaki köprüdür. **Spring Data JPA** framework'ünün `JpaRepository` arayüzünden türetilen Repository arayüzleri; CRUD operasyonlarını otomatik olarak sağlar. Geliştirici yalnızca metot ismi tabanlı türetilmiş sorguları (`findByPlakaNo`, `findByMarkaIgnoreCase`) ya da özel JPQL sorgularını (`@Query`) tanımlar; Spring Data JPA bu arayüzlerin çalışma zamanında otomatik olarak implementasyonunu üretir.

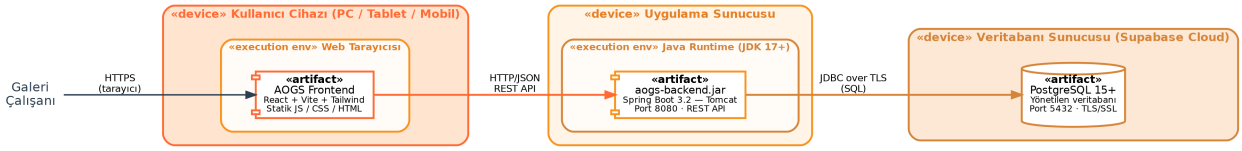
AOGS'taki yedi Repository:

- **AracRepository:** Plaka ile arama, marka/durum bazlı filtreleme, stok değeri hesaplama.
- **MusteriRepository:** TC kimlik ile arama, ad bazlı içerme sorguları.
- **SatisRepository:** Tarih aralığı, müşteri, çalışan bazlı filtreleme; aylık satış agregasyonları, kâr ve ciro hesaplamaları.
- **EkspertizRepository:** Araç bazlı ekspertiz geçmişi, en son ekspertiz erişimi.
- **CalisanRepository:** Çalışan listesi, performans sorguları.
- **SigortaRepository:** Yaklaşan ve süresi dolmuş sigortalar.
- **MuayeneRepository:** Yaklaşan ve süresi dolmuş muayeneler.

Tüm Repository sorguları, veritabanı tarafında B-Tree indeksleri kullanılarak çalıştırılır. Bu sayede milyonlarca satır içeren tablolarda dahi milisaniyelik yanıt süreleri elde edilir.

2.2. Donanım / Yazılım Eşlemesi

AOGS, üç farklı fiziksel/sanal node üzerinde çalışmak üzere tasarlanmıştır: **İstemci Cihazı** (kullanıcının PC, tablet veya mobil cihazı), **Uygulama Sunucusu** (Spring Boot backend) ve **Veritabanı Sunucusu** (Supabase tarafından yönetilen PostgreSQL). Tüm iletişim ağ üzerinden HTTPS ve TLS şifrelemesi ile yapılır.



Şekil 2.2: *AOGS Konuşlanma (Deployment) Diyagramı — Üç ayrı node üzerindeki bileşenlerin dağılımı.*

Sunucu Donanım Gereksinimleri

Bileşen	Minimum	Önerilen
Uygulama Sunucusu CPU	2 vCPU	4 vCPU
Uygulama Sunucusu RAM	2 GB	4 GB
Uygulama Sunucusu Disk	10 GB SSD	20 GB SSD
Veritabanı (Supabase Tier)	Free / Hobby	Pro Plan
Ağ Bağlantısı	100 Mbps	1 Gbps
İstemci Cihaz	Modern web tarayıcısı (Chrome 90+, Firefox 88+, Safari 14+, Edge 90+)	-

Tablo 2.1: *Sunucu Donanım Gereksinimleri*

Yazılım Bağımlılıkları

AOGS'un geliştirme ve çalışma ortamında ihtiyaç duyduğu yazılım bileşenleri iki gruba ayrılır: **Backend** ve **Frontend**.

Bileşen	Sürüm	Rol
BACKEND		
Java Development Kit	17 LTS+	JVM çalışma ortamı
Spring Boot	3.2	Web framework, REST API
Spring Data JPA	3.2	Repository katmanı
Hibernate	6.4	ORM (Object-Relational Mapping)
PostgreSQL Driver	42.7	JDBC sürücüsü
Apache Maven	3.8+	Derleme aracı
FRONTEND		

Node.js	18+	Geliştirme runtime
React	18.3	UI bileşen kütüphanesi
Vite	5.3	Geliştirme sunucusu, build aracı
Tailwind CSS	3.4	Utility-first stil framework'ü
axios	1.7	HTTP istemcisi
React Router	6.23	İstemci tarafı yönlendirme
Recharts	2.12	Raporlama grafikleri
VERİTABANI		
PostgreSQL	15+	İlişkisel veritabanı (Supabase yönetiyor)

Tablo 2.2: *Yazılım Bağımlılıkları (Backend / Frontend / Veritabanı)*

Dağıtım (Deployment) Modeli

Üretim ortamında her bileşen ayrı bir host üzerinde konuşlanır:

- **Frontend:** Vite ile derlenmiş statik dosyalar (HTML, CSS, JS) Nginx, Apache, Vercel, Netlify ya da bir CDN üzerinde servis edilir. Statik dosyalar olduğu için hızlı yüklenir ve global olarak dağıtılabilir.
- **Backend:** Maven ile üretilen `aogs-backend.jar` dosyası, JDK 17 kurulu bir sunucuda çalıştırılır. Spring Boot, gömülü Tomcat ile varsayılan olarak 8080 portu üzerinden hizmet verir.
- **Veritabanı:** Supabase platformunda yönetilen bir PostgreSQL örneği kullanılır. Bağlantı bilgileri, backend'in `application.properties` dosyasında ya da ortam değişkenlerinde tutulur.

> **İPUCU**
>
> Geliştirme ortamında üç bileşen aynı makinede çalışabilir: frontend Vite dev sunucusu (5173), backend Spring Boot (8080) ve yerel PostgreSQL ya da hızlı geliştirme için H2 in-memory veritabanı. Vite proxy yapılandırması, frontend'den gelen `/api/*` isteklerini backend'e yönlendirir; bu sayede CORS yapılandırması geliştirme aşamasında gereksizdir.

2.3. Kalıcı Veritabanı Yönetimi

AOGS tüm verilerini **PostgreSQL** ilişkisel veritabanında kalıcı olarak saklar. Üretim ortamında **Supabase** bulut platformu tercih edilmektedir. Bu seçim aşağıdaki avantajları sağlar:

- **ACID Garantileri:** Atomicity, Consistency, Isolation, Durability özellikleri sayesinde finansal veriler (satış, kâr, fatura) güvenle saklanır.
- **B-Tree İndeksleme:** Sık sorgulanan sütunlar (plaka, TC kimlik, satış tarihi) üzerinde otomatik B-Tree indeksleri oluşturulur. Karmaşıklık O(log n)'dir.

- **Otomatik Yedekleme:** Supabase günlük otomatik yedek alır; felaket kurtarma senaryolarında veri kaybı minimize edilir.
- **Yüksek Erişilebilirlik:** Yönetilen veritabanı altyapısı %99.9+ uptime garantisi sunar.
- **SQL Standardı:** İhtiyaç halinde başka bir PostgreSQL sağlayıcısına (AWS RDS, Google Cloud SQL, yerel PostgreSQL) geçiş yalnızca bağlantı dizgisi değişikliği ile mümkündür.

ORM Eşleme: JPA + Hibernate

Java nesnelerinin veritabanı tablolarına eşlenmesi için **Java Persistence API (JPA)** standardı ve onun en yaygın implementasyonu olan **Hibernate** kullanılır. JPA Entity'ler (`@Entity`) doğrudan veritabanı tablolarına karşılık gelir; geliştirici düşük seviyeli SQL yazmak zorunda kalmaz.

Araç hiyerarşisi gibi özel durumlarda `@Inheritance(strategy = InheritanceType.JOINED)` stratejisi kullanılmıştır. Bu stratejide ortak alanlar (`araclar` tablosu) ve alt sınıfa özgü alanlar (`otomobiller`, `suvler`, `ticariler`, `motosikletler` tabloları) ayrı tablolarda normalize edilir; Hibernate sorgularda otomatik olarak JOIN üretir. Bu yaklaşım hem OOP hiyerarşisini korur hem de veritabanı normalizasyonunu sağlar.

İndeks Stratejisi

Performans için kritik sütunlar üzerinde aşağıdaki indeksler tanımlanmıştır:

Tablo	Sütun	İndeks Türü	Amaç
araclar	plaka_no	B-Tree, UNIQUE	Plaka ile hızlı arama; tekrarlı kayıt engelleme
araclar	marka	B-Tree	Marka bazlı filtreleme
araclar	durum	B-Tree	SATISTA / SATILDI filtreleme
musteriler	tc_kimlik	B-Tree, UNIQUE	TC kimlik ile hızlı müşteri arama
satislar	satis_tarihi	B-Tree	Tarih aralığı sorguları, raporlama
satislar	arac_id, musteri_id, calisan_id	B-Tree (FK)	İlişkili kayıtlara hızlı erişim
ekspertizler	arac_id	B-Tree (FK)	Bir aracın tüm ekspertiz geçmişi

Tablo 3.2: *AOGS Veritabanı İndeks Stratejisi (önizleme; ER diyagramı için Bölüm 3.3'e bakınız)*

2.4. Erişim Kontrolü ve Güvenlik

AOGS, oto galeri çalışanlarının kullanımına yönelik bir intranet uygulamasıdır. Bu sürümde tek tip kullanıcı (galeri çalışanı) tanımlıdır; rol bazlı erişim kontrolü (RBAC) ileri sürümlere bırakılmıştır. Bununla birlikte; veri bütünlüğünün korunması, uygun kullanımın sağlanması ve olası kötü kullanım senaryolarının engellenmesi için aşağıdaki güvenlik kontrolleri tasarıma dahil edilmiştir:

- **Çift Katmanlı Doğrulama:** Her form alanı hem React tarafında (istemci) hem de Spring Validation ile (sunucu) doğrulanır. Sunucu, istemciden gelen veriye asla güvenmez.
- **SQL Injection Koruması:** Tüm veritabanı sorguları JPA üzerinden parametreli olarak çalıştırılır. Ham SQL stringi concatenation hiçbir yerde kullanılmaz.
- **CORS Politikası:** Backend, yalnızca güvenilir domain'lerden gelen istekleri kabul eder (`CorsConfig`).
- **HTTPS Şifrelemesi:** Üretim ortamında tüm istemci-sunucu iletişimi TLS ile şifrelenir.
- **İş Kuralı Doğrulamaları:** Service katmanında; satılmış araçların silinememesi, mükerrer plaka kayıtlarının engellenmesi gibi iş kuralları uygulanır.

REST API İşlev Yetki Matrisi

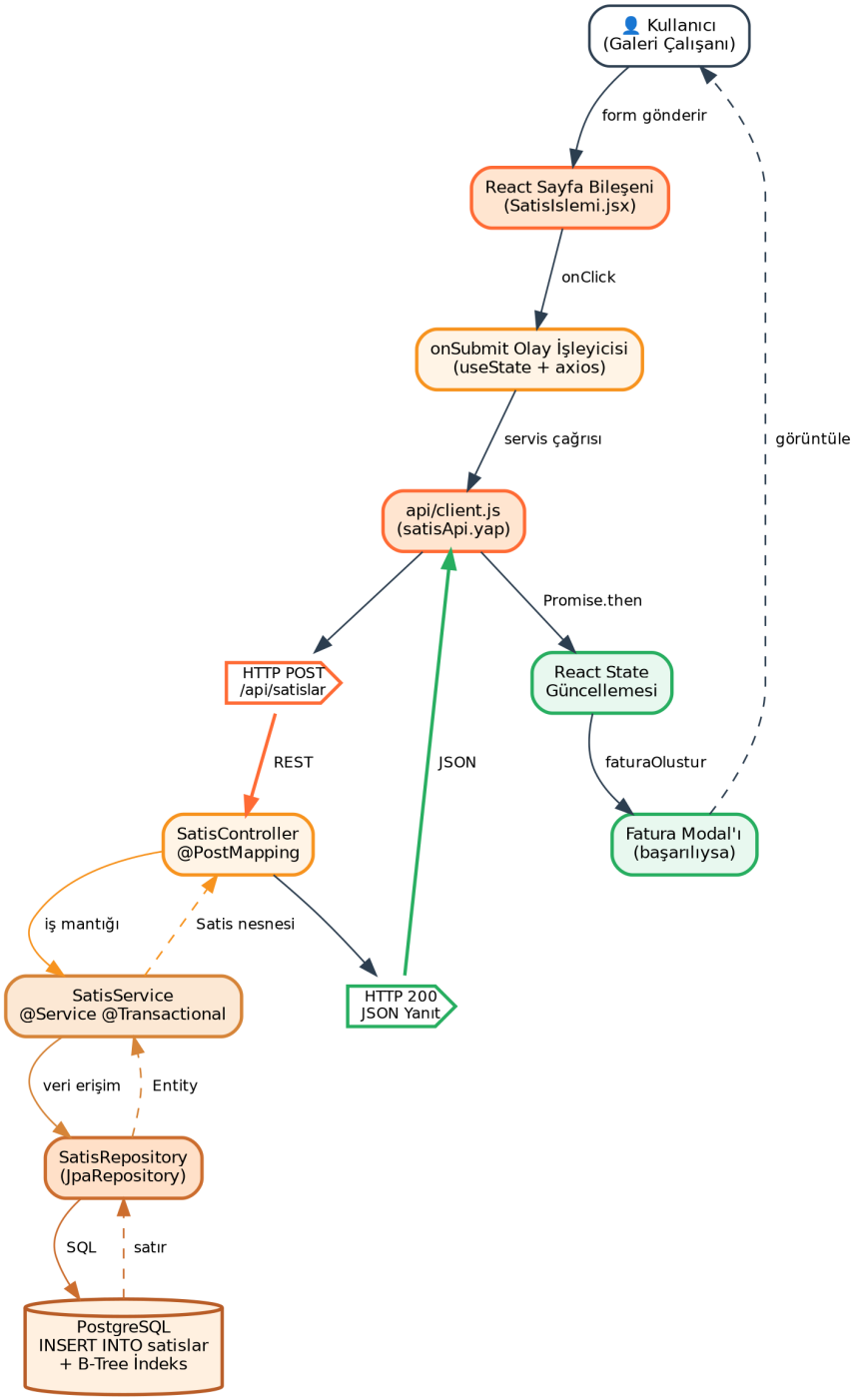
İşlev	HTTP Endpoint	Ön Koşul / İş Kuralı
Araç Ekleme	`POST /api/araclar`	Plaka benzersiz, fiyatlar pozitif, zorunlu alanlar dolu
Araç Silme	`DELETE /api/araclar/{id}`	Araç durumu SATILDI değilse
Müşteri Kayıt	`POST /api/musteriler`	TC kimlik 11 haneli ve benzersiz
Satış Gerçekleştirme	`POST /api/satislar`	Araç SATISTA durumda, müşteri kayıtlı, atomik işlem
Satış İptali	`POST /api/satislar/{id}/iptal`	Satış 24 saat içinde yapılmışsa
Ekspertiz Oluşturma	`POST /api/ekspertizler`	Araç kayıtlı, çalışan kayıtlı
Raporlama	`GET /api/raporlar/*`	Salt-okunur, herhangi bir kısıt yok

Tablo 2.3: *AOGS REST API İşlev Yetki Matrisi*

> **UYARI**
>
> İleri sürümlerde **JWT (JSON Web Token)** tabanlı kimlik doğrulama ve **Spring Security** ile rol bazlı erişim kontrolü (Yönetici, Satış Personeli, Ekspertiz Uzmanı) eklenmesi planlanmaktadır.

2.5. Küresel Kontrol Akışı

AOGS, **olay güdümlü (event-driven) ve istek-yanıt (request-response)** kontrol akışını birlikte benimser. Frontend tarafında React'in olay yakalama mekanizması; backend tarafında ise Spring MVC'nin DispatcherServlet üzerinden HTTP istek yönlendirme mekanizması bu modelin temelini oluşturur. Kullanıcı bir butona tıkladığında React, ilgili olay işleyiciyi (event handler) tetikler; bu handler axios üzerinden REST API'ye HTTP isteği gönderir; backend isteği işler, veritabanı operasyonunu gerçekleştirir ve JSON yanıtı döner.



Şekil 2.4: *AOGS Küresel Kontrol Akışı — Satış işlemi senaryosunda kullanıcı etkileşiminden veri güncellemesine kadar olan tam akış.*

Merkezi Olmayan Kontrol Yapısı

AOGS'ta merkezi bir kontrol nesnesi yerine **merkezi olmayan (decentralized)** bir kontrol yapısı tercih edilmiştir. Her Controller, Service ve Repository yalnızca kendi sorumluluk alanından sorumludur. Bu yaklaşımın avantajları:

- **Modülerlik:** Her endpoint, servis ve repository bağımsız olarak geliştirilebilir ve test edilebilir.
- **Ölçeklenebilirlik:** Yeni bir endpoint eklemek mevcut kontrol akışını değiştirmez.
- **Okunabilirlik:** Her sınıfın sorumluluk alanı dar ve net tanımlıdır (Tek Sorumluluk İlkesi).

- **Paralel Geliştirme:** Altı kişilik ekibin farklı modüller üzerinde eşzamanlı çalışabilmesini kolaylaştırır.

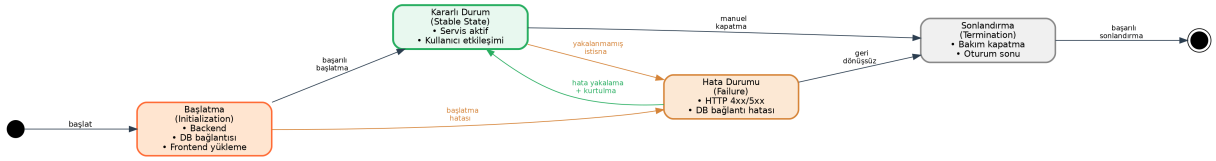
Örnek Akış: Satış Gerçekleştirme

Bir satış işleminin küresel kontrol akışı şu adımlardan oluşur:

1. Kullanıcı, Satış İşlemi sayfasında formu doldurur ve 'Satışı Tamamla' butonuna tıklar.
2. React'in onSubmit olay işleyicisi tetiklenir; form verileri bir DTO nesnesine dönüştürülür.
3. axios üzerinden `POST /api/satislar` isteği gönderilir; HTTP gövdesinde JSON formatında DTO bulunur.
4. Spring Boot'un DispatcherServlet, isteği SatisController'ın `@PostMapping` metoduna yönlendirir.
5. Controller, gelen DTO'yu doğrular ve `SatisService.satisYap(...)` metodunu çağırır.
6. Service `@Transactional` ile atomik bir işlem başlatır: araç ID ile araç bulunur, müşteri TC ile müşteri bulunur, yeni Satis Entity oluşturulur.
7. `SatisRepository.save(satis)` çağrısıyla yeni satış INSERT edilir; `aracRepository` üzerinden aracın durumu SATILDI'ya UPDATE edilir.
8. İşlem başarılıysa @Transactional COMMIT yapar; kaydedilen Satis nesnesi Controller'a döner.
9. Controller, Spring'in HttpMessageConverter'ı (Jackson) ile Satis nesnesini JSON'a çevirir; HTTP 200 yanıtı olarak döner.
10. Frontend'de axios Promise resolved olur; React state güncellenir, satış tablosu yeniden render edilir, fatura modal'ı açılır.

2.6. Sınır Koşulları

Sistemin olağandışı durumlarda (başlatma, sonlandırma, hata) nasıl davranacağı alt bölümler halinde ele alınmıştır.



Şekil 2.3: *AOGS Sınır Koşulları Durum Diyagramı*

2.6.1. Başlatma (Initialization)

AOGS başlatma süreci üç bileşen için paralel olarak yürür:

1. **Veritabanı:** Supabase üzerindeki PostgreSQL örneği her zaman çalışır durumdadır; ek başlatma adımına gerek yoktur.
2. **Backend (Spring Boot):** JVM `aogs-backend.jar` dosyasını çalıştırır; Spring Boot ApplicationContext başlatılır, tüm `@Component`, `@Service`, `@Repository`, `@Controller` bean'leri otomatik bulunur ve enjekte edilir; Hibernate'in EntityManagerFactory'si oluşturulur, veritabanı bağlantı havuzu (HikariCP) açılır; gömülü Tomcat 8080 portunda dinlemeye başlar.

3. **Frontend (React):** Kullanıcı tarayıcıda uygulama URL'sine gittiğinde Vite tarafından üretilmiş `index.html` ve JS bundle'ları yüklenir; React DOM'u monte eder; React Router varsayılan rotaya (/ — Dashboard) yönlendirir.

2.6.2. Sonlandırma (Termination)

Sonlandırma da yine üç bileşen için ayrı düşünülür:

- **Frontend:** Kullanıcı tarayıcı sekmesini kapattığında React uygulaması bellekten silinir. Server-side bir oturum tutulmadığı için temiz bir kapanış olur.
- **Backend:** Bakım ya da güncelleme amacıyla kontrollü kapatma işlemleri Spring Boot'un graceful shutdown özelliği ile yapılır; sürmekte olan istekler tamamlanır, yeni istekler reddedilir.
- **Veritabanı:** Aktif bağlantı havuzu kapatılır, açık transaction'lar commit ya da rollback edilir.

2.6.3. Hata Durumları (Failure)

AOGS'ta hatalar üç kategoride ele alınır:

Beklenen İş Hataları

Geçersiz girdi, mükerrer TC, bulunamayan araç gibi hatalar `IsKuraliException` türetilen özel istisnalar olarak fırlatılır. `@RestControllerAdvice` ile tanımlanan Global Exception Handler bu istisnaları yakalar ve kullanıcıya HTTP 400 ya da 422 ile birlikte anlamlı bir mesaj döner. Frontend'de axios bu yanıtı yakalayıp toast bildirimi olarak gösterir.

Beklenen Sistem Hataları

Veritabanı bağlantısının geçici kesilmesi, ağ hatası gibi durumlarda Spring HTTP 503 (Service Unavailable) döner. Frontend kullanıcıya 'sunucuya ulaşılamıyor, bir süre sonra tekrar deneyin' mesajı gösterir.

Beklenmeyen Sistem Hataları

Yakalanmamış istisnalar (NullPointerException, OutOfMemoryError gibi) Spring'in varsayılan hata sayfasıyla HTTP 500 olarak döner. Stack trace loglanır; kullanıcıya teknik detay gösterilmez. Üretimde APM araçları (Spring Boot Actuator, Sentry) ile hatalar geliştirici ekibine bildirilir.

3. ALT DÜZEY TASARIM

Bu bölüm AOGS'un sınıf, paket ve veritabanı düzeyindeki tasarımını detaylı biçimde sunmaktadır. Üst düzey mimariye ek olarak; her sınıfın sorumluluğu, JPA Entity eşlemeleri, Repository sorgu metotları, Service iş mantığı, Controller endpoint tanımları, veritabanı şeması ve uygulanan tasarım desenleri birlikte açıklanmaktadır.

3.1. Nesne Tasarım Kararları

Tasarım sürecinde; okunabilirlik, yeniden kullanılabilirlik, performans ve bakım kolaylığı arasındaki ödünleşimler göz önünde bulundurularak kritik kararlar verilmiştir. Bu kararlar aşağıda gerekçeleri ile açıklanmaktadır.

3.1.1. JPA Entity'leri ile ORM Eşlemesi

Java sınıfları ile veritabanı tabloları arasındaki eşleme için **Java Persistence API (JPA)** standardı ve onun en yaygın implementasyonu olan **Hibernate** tercih edilmiştir. Düz JDBC ile manuel SQL yazmak yerine bu yaklaşım seçilmesinin gerekçeleri:

- **Geliştirici verimliliği:** Tek bir sınıf tanımı hem domain modelini hem de veritabanı şemasını ifade eder.
- **Tip güvenliği:** SQL stringleri yerine Java metotları kullanılır; yazım hataları derleme zamanında yakalanır.
- **Veritabanı bağımsızlığı:** Aynı kod PostgreSQL, MySQL, H2 gibi farklı veritabanlarında çalışabilir.
- **Otomatik şema oluşturma:** Geliştirme sırasında `spring.jpa.hibernate.ddl-auto` ayarıyla şema otomatik üretilebilir; üretimde Flyway/Liquibase ile sürüm yönetimi mümkündür.
- **İlişki yönetimi:** `@ManyToOne`, `@OneToMany` gibi anotasyonlar ile varlıklar arası ilişkiler net biçimde modellenir.

3.1.2. Spring Dependency Injection ile Bean Yönetimi

Service, Repository ve Controller sınıfları arasındaki bağımlılıklar manuel olarak yönetilmek yerine; Spring IoC Container'ın **Dependency Injection** mekanizması ile otomatik olarak çözülür. Her bean varsayılan olarak Singleton scope'unda yönetilir; framework, sınıfın constructor parametrelerini görerek uygun bağımlılıkları enjekte eder. Bu tercihin avantajları:

- **Tek yapılandırma noktası:** Bean'lerin yaşam döngüsü ve scope'u Spring tarafından merkezi olarak yönetilir.
- **Test edilebilirlik:** Constructor injection sayesinde mock nesneler kolayca yerleştirilebilir; Service'ler birim testlerinde Repository olmadan test edilebilir.
- **Gevşek bağlılık:** Sınıflar somut implementasyonlara değil, arayüzlere bağımlı olabilir.
- **Aspect-Oriented Programming desteği:** `@Transactional`, `@Cacheable` gibi cross-cutting concern'ler bean proxy'leri ile transparan biçimde uygulanır.

3.1.3. Sunum-Mantık Ayrımı (Frontend / Backend)

Sunum katmanı (React) ile iş mantığı katmanı (Spring Boot) ayrı süreçler olarak çalıştırılarak **fiziksel bir ayırma** sağlanmıştır. Bu ayrımın getirdiği avantajlar:

- **Ayrı dağıtım:** Frontend ve backend bağımsız olarak deploy edilebilir; birinin güncellenmesi diğerini etkilemez.
- **Yeniden kullanım:** Aynı REST API'yi farklı istemciler (mobil uygulama, üçüncü taraf entegrasyonlar) kullanabilir.
- **Teknoloji bağımsızlığı:** Frontend gelecekte React Native, Vue ya da başka bir framework'e taşınabilir; backend etkilenmez.
- **Ölçeklendirme:** Yoğun trafikte backend yatay olarak ölçeklenebilir (birden fazla instance), frontend ise CDN ile global olarak dağıtılabilir.

3.1.4. Soyut Sınıflar ve Polimorfik Davranış

AOGS'ta kalıtım hiyerarşileri için soyut sınıflar kullanılmıştır:

- `Arac` sınıfı `@Entity` ile işaretlenmiş soyut bir sınıftır; `@Inheritance(strategy = JOINED)` stratejisiyle her alt sınıfı (Otomobil, SUV, Ticari, Motosiklet) ayrı bir tabloya eşler.
- `Kisi` sınıfı ise `@MappedSuperclass` olarak tanımlanmıştır; ortak alanları (ad, soyad, TC kimlik) sağlar ancak kendisi bir tablo oluşturmaz. Müşteri ve Çalışan kendi tablolarına sahiptir.
- Bu iki yaklaşım, hangi kalıtım stratejisinin domain'e uygun olduğuna göre tercih edilmiştir.

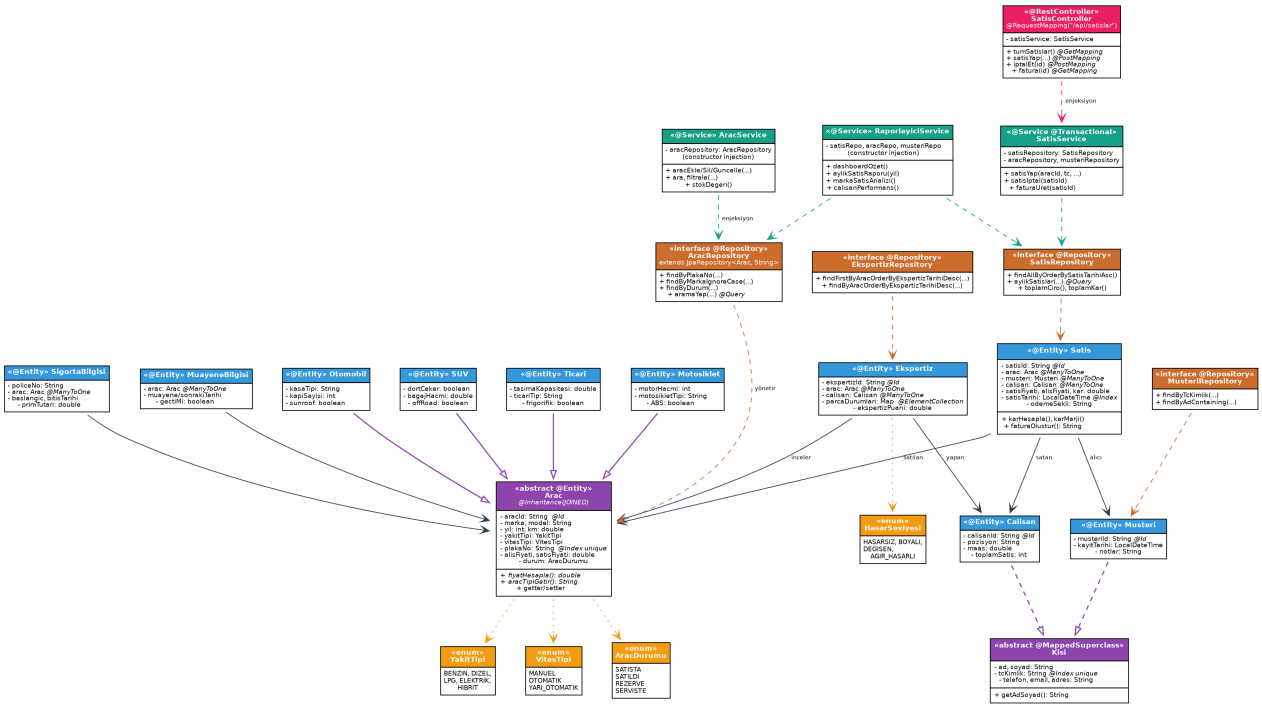
3.1.5. Polimorfik Fiyat Hesaplama

Her araç tipinin satış fiyatı farklı iş kuralları içerir (örneğin sunroof'lu otomobil +5%, dört çeker SUV +8%, frigorifik ticari +12% premium). Bu ihtiyaç; `Arac` soyut sınıfında soyut `fiyatHesapla()` metodu ile çözülmüştür. Her alt sınıf kendi hesaplama mantığını override ederek **polimorfik davranış** sergiler. Yeni bir araç tipi eklendiğinde mevcut kod değiştirilmez (Açık-Kapalı İlkesi).

3.2. Son Nesne Tasarımı (UML Sınıf Diyagramı)

Aşağıdaki UML sınıf diyagramı, AOGS'un tüm Entity, Repository, Service ve Controller sınıflarını; aralarındaki ilişkileri ve JPA anotasyonlarını birlikte gösterir. Diyagramda sınıflar kategorilere göre renk kodlanmıştır:

- **Mor:** Soyut sınıflar (`Arac`, `Kisi`)
- **Mavi:** Concrete JPA Entity'leri
- **Sarı/Turuncu:** Enumerasyonlar
- **Koyu Turuncu:** Repository arayüzleri (Spring Data JPA)
- **Turkuaz:** Service sınıfları
- **Pembe:** Controller sınıfları (REST API)



Şekil 3.2: *AOGS Tam UML Sınıf Diyagramı — JPA Entity'leri, Repository, Service, Controller katmanları ve aralarındaki ilişkiler.*

3.2.1. İlişki Türleri

Diyagramdaki ilişki türleri:

İlişki	Sembol	Anlamı
Kalıtım (Inheritance)	━▷ (boş üçgen, mor)	Alt sınıf üst sınıfın özelliklerini ve davranışını miras alır
İlişkilendirme (FK)	━▶ (ok, koyu)	Bir entity, başka bir entity'ye foreign key ile bağlıdır
Repository → Entity	┅▷ (kesikli, turuncu)	Repository ilgili entity'yi yönetir
Service → Repository	┅▷ (kesikli, turkuaz)	Spring Constructor Injection ile bağımlılık
Controller → Service	┅▷ (kesikli, pembe)	Spring Constructor Injection ile bağımlılık
Enum Kullanımı	┅▷ (noktalı, sarı)	Sabit değer kümelerinin tip-güvenli temsili

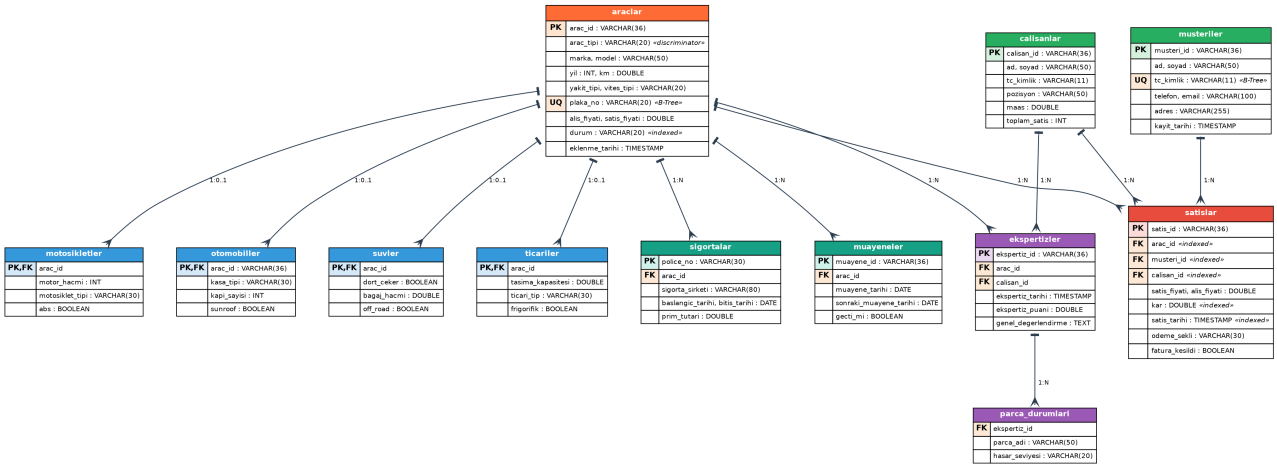
3.2.2. Temel İlişkilerin Tasarım Gerekçeleri

Sınıflar	İlişki Tipi	Çokluluk	Tasarım Gerekçesi
`Arac — Otomobil/SUV/Ticari/Motosiklet`	Kalıtım (JOINED)	1 ← 4	Ortak alanları araclar tablosunda tutarken alt sınıf alanlarını ayrı tablolarda normalize etmek.
`Kisi — Musteri/Calisan`	Kalıtım (MappedSuperclass)	1 ← 2	Ortak alanları paylaşmak; her alt sınıf kendi tablosuna sahip olur.
`Satis — Arac, Musteri, Calisan`	@ManyToOne (FK)	* → 1	Her satış bir araç, müşteri ve çalışana bağlıdır; FK'ler indeks ile hızlı sorgulanır.
`Ekspertiz — Arac`	@ManyToOne (FK)	* → 1	Bir araç için birden fazla ekspertiz olabilir; her ekspertiz tek araca bağlıdır.
`Ekspertiz — parcaDurumlari`	@ElementCollection	1 ◆ *	Parça-hasar eşleştirmeleri ayrı bir tabloda tutulur.
`Repository — JpaRepository`	extends	1 ▷ 1	Spring Data JPA otomatik CRUD operasyonları sağlar.
`Service — Repository`	Constructor DI	1 → N	Service, ihtiyacı olan repository'leri Spring tarafından enjekte alır.

Tablo 3.1: *AOGS Sınıf İlişkileri ve Tasarım Gerekçeleri*

3.3. Veritabanı Şeması (ER Diyagramı)

AOGS verileri PostgreSQL veritabanında 12 tabloda saklanır. JPA Entity hiyerarşisi, JOINED kalıtım stratejisi ile veritabanına yansıtılır: ortak alanlar ana `araclar` tablosunda, alt sınıfa özgü alanlar ayrı tablolarda (`otomobiller`, `suvler`, `ticariler`, `motosikletler`) bulunur. Her alt tablo, ana tabloya foreign key ile bağlıdır.



Şekil 3.3: *AOGS Varlık-İlişki (ER) Diyagramı — PostgreSQL şeması; 12 tablo, foreign key ilişkileri ve indekslenmiş alanlar.*

3.3.1. Tablo Açıklamaları

- `araclar`**:** Tüm araçların ortak bilgilerini içerir. `arac_tipi` discriminator sütunu ile alt sınıfı belirler.
- `otomobiller, suvler, ticariler, motosikletler`**:** Her biri ana `araclar` tablosuna FK ile bağlı; alt sınıfa özgü alanları (kasaTipi, dortCeker, frigorifik, motorHacmi vb.) tutar.
- `musteriler`**:** Müşterilerin tüm bilgilerini içerir; `tc_kimlik` üzerinde unique B-Tree indeksi vardır.
- `calisanlar`**:** Galeri çalışanları; performans (toplamSatis), maaş, pozisyon bilgileri.
- `satislar`**:** Gerçekleşen tüm satışlar; arac_id, musteri_id, calisan_id FK'leri ile ilgili kayıtlara bağlanır. `satis_tarihi` indekslidir.
- `ekspertizler`**:** Ekspertiz raporları; `ekspertiz_puani` alanı raporlama için kullanılır.
- `parca_durumlari`**:** Her ekspertiz için parça bazlı hasar kaydı (@ElementCollection eşlemesi).
- `sigortalar, muayeneler`**:** Aracın sigorta ve muayene tarihlerini takip eder; bildirim sistemi bu tabloları periyodik olarak sorgular.

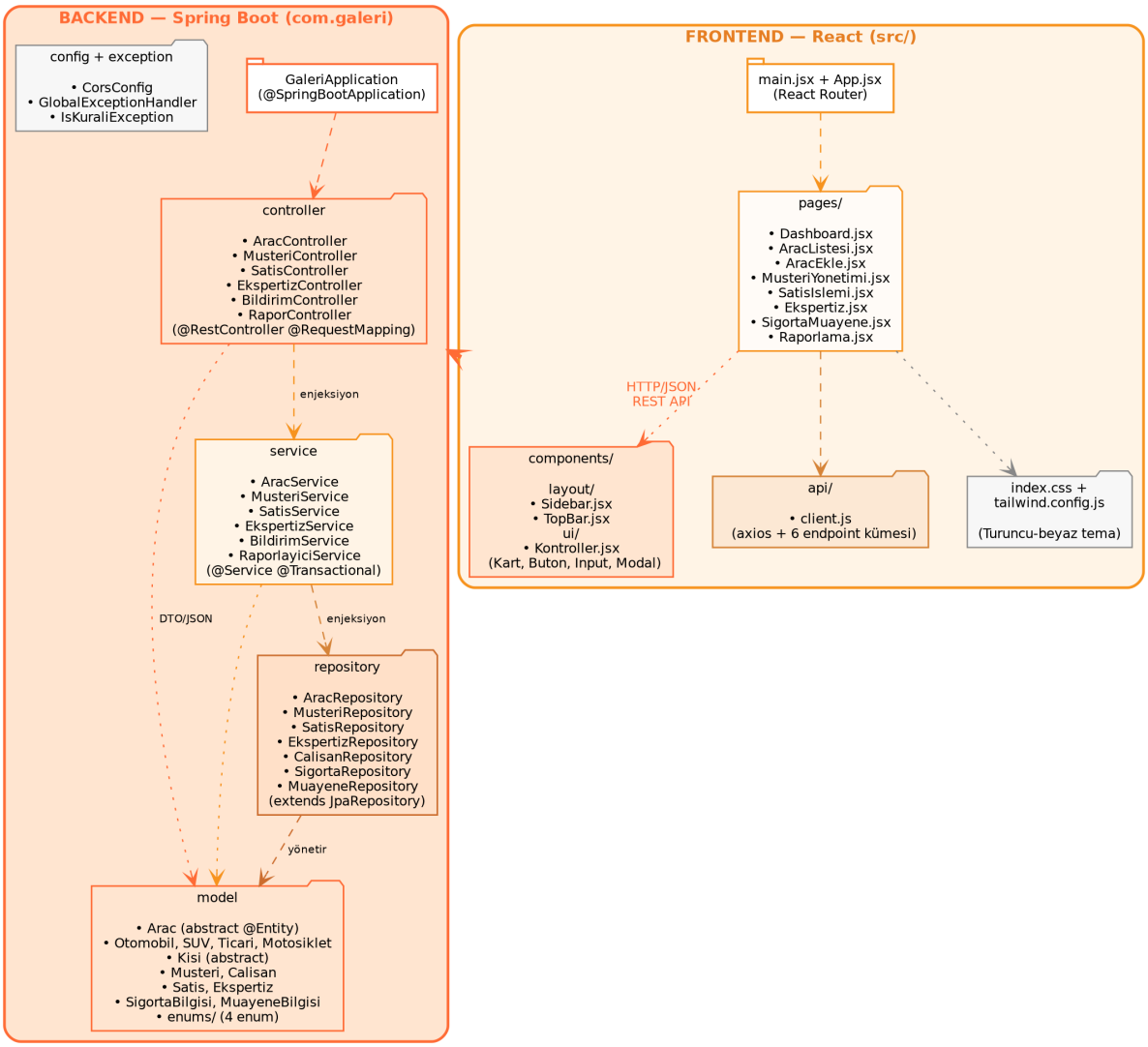
3.3.2. Veri Bütünlüğü Kuralları

Veritabanı seviyesinde aşağıdaki kısıtlamalar uygulanır:

- **Primary Key (PK):** Her tabloda benzersiz kimlik. UUID stringi (36 karakter) tercih edilir; bu sayede ID üretimi merkezi bir sequence'a bağlı olmaz.
- **Foreign Key (FK):** Tutarsız kayıtların engellenmesi için satislar.arac_id, satislar.musteri_id gibi tüm referanslar veritabanı seviyesinde FK ile korunur.
- **UNIQUE Kısıtlamaları:** `araclar.plaka_no` ve `musteriler.tc_kimlik` üzerinde tekil kısıt vardır.
- **NOT NULL:** Zorunlu alanlar (marka, model, ad, soyad gibi) NULL kabul etmez.
- **CHECK Kısıtlamaları:** Fiyatlar negatif olamaz, yıl makul bir aralıkta olmalıdır.

3.4. Paketler / Modül Yapısı

AOGS kaynak kodu; backend (Java) ve frontend (JavaScript) olmak üzere iki ayrı modül halinde organize edilmiştir. Her bir tarafın kendi paket/dizin yapısı vardır ve aralarında REST API üzerinden iletişim kurulur.



Şekil 3.1: *AOGS Paket Diyagramı — Backend (`com.galeri.*`) ve Frontend (`src/*`) modülleri ile aralarındaki REST API bağlantısı.*

3.4.1. Backend Paket Yapısı (com.galeri.*)

Spring Boot backend uygulaması `com.galeri` kök paketi altında; her katman için ayrı bir alt paket içerir. Bu yapılanma; Spring'in component scanning'i için optimal düzendir ve klasik 'package-by-layer' yaklaşımını yansıtır.

Paket	Sorumluluk	İçerdiği Sınıflar
`com.galeri`	Uygulama giriş noktası	GaleriApplication (@SpringBootApplication)

`com.galeri.model`	İş domaininin JPA Entity'leri ve enum'lar	Arac (abstract @Entity), Otomobil, SUV, Ticari, Motosiklet, Kisi (@MappedSuperclass), Musteri, Calisan, Satis, Ekspertiz, SigortaBilgisi, MuayeneBilgisi + 4 enum
`com.galeri.repository`	Veri erişim katmanı (Spring Data JPA)	AracRepository, MusteriRepository, SatisRepository, EkspertizRepository, CalisanRepository, SigortaRepository, MuayeneRepository (extends JpaRepository)
`com.galeri.service`	İş mantığı katmanı	AracService, MusteriService, SatisService, EkspertizService, BildirimService, RaporlayiciService (@Service @Transactional)
`com.galeri.controller`	REST API uç noktaları	AracController, MusteriController, SatisController, EkspertizController, BildirimController, RaporController (@RestController @RequestMapping)
`com.galeri.config`	Spring yapılandırma sınıfları	CorsConfig (@Configuration)
`com.galeri.exception`	Özel istisna sınıfları ve global handler	IsKuraliException, GlobalExceptionHandler (@RestControllerAdvice)

Tablo 3.3: *Backend Paket Yapısı ve Sorumluluklar*

3.4.2. Frontend Modül Yapısı (src/*)

React frontend uygulaması, modern bir SPA yapısında `src/` dizini altında organize edilmiştir. Her klasör belirli bir sorumluluğa atanmıştır:

Dizin / Dosya	Sorumluluk	İçerdikleri
`main.jsx`	Uygulama giriş noktası	React DOM'u monte eder; App bileşenini render eder
`App.jsx`	Yönlendirme (Routing)	React Router yapılandırması; sayfalar arası rotalar
`pages/`	Sayfa bileşenleri	Dashboard, AracListesi, AracEkle, MusteriYonetimi, SatisIslemi, Ekspertiz, SigortaMuayene, Raporlama (.jsx)
`components/layout/`	Sayfa düzeni bileşenleri	Sidebar.jsx (sol navigasyon), TopBar.jsx (üst başlık)
`components/ui/`	Yeniden kullanılabilir UI bileşenleri	Kart, Buton, Input, Modal, IkonluKart vb. (Kontroller.jsx)
`api/`	Backend ile iletişim	client.js (axios; aracApi, musteriApi, satisApi, ekspertizApi, bildirimApi, raporApi)

`index.css`	Genel stiller	Tailwind direktifleri (@tailwind base/components/utilities)
`tailwind.config.js`	Tailwind yapılandırması	Özel renkler (aogs.*), font ailesi, plugin'ler

Tablo 3.4: *Frontend Modül Yapısı*

Paket Bağımlılık Kuralları

Backend paketleri arasındaki bağımlılıklar tek yönlü tutulmuştur:

- `model` → hiçbir şeye bağımlı değildir (en alt katman).
- `repository` → yalnızca `model`'e bağımlıdır.
- `service` → `repository, model, exception`'a bağımlıdır.
- `controller` → `service, model`'e bağımlıdır.
- `config` ve `exception` → bağımsız yardımcılardır.

> **BİLGİ**
>
> Döngüsel bağımlılıklar (circular dependencies) yasaktır. Bu kural, Spring'in bean enjeksiyon mekanizmasını sorunsuz işletmek ve modülerliği korumak için kritiktir.

3.5. Sınıf Arayüzleri

Bu bölümde AOGS'un sınıfları kategori bazlı olarak (Model, Repository, Service, Controller, Frontend) detaylı sınıf arayüzü formatında belgelendirilmiştir. Her sınıf için kısa açıklama, öznitelikler tablosu ve metotlar tablosu sunulmaktadır. Erişim belirleyicileri: **+** public, **-** private, **#** protected, **~** package-private.

3.5.1. Model Katmanı (JPA Entity'leri)

Model katmanı, iş domaininin dijital yansımalarını barındıran JPA Entity sınıflarından oluşur. Her sınıf, `@Entity` anotasyonu ile bir veritabanı tablosuna eşlenir. Tüm alanlar `private` ya da `protected` olup, erişim getter/setter metotları üzerinden yapılır (kapsülleme).

3.5.1.1. `Arac` *«abstract @Entity»*

Tüm araç tiplerinin ortak özniteliklerini ve davranışlarını tanımlayan soyut JPA Entity'sidir. `@Inheritance(strategy = InheritanceType.JOINED)` stratejisi ile veritabanında ayrı tablolar olarak normalize edilir; alt sınıflar (`Otomobil`, `SUV`, `Ticari`, `Motosiklet`) her biri kendi tablosuna sahiptir ve `araclar` tablosuna foreign key ile bağlanır.

Öznitelikler:

Alan	Tip	Açıklama
`aracId`	`String`	@Id - Birincil anahtar; UUID formatında.
`marka`	`String`	Aracın markası (Toyota, Ford vb.). @Column(nullable = false).
`model`	`String`	Aracın modeli (Corolla, Civic vb.).
`yil`	`int`	Üretim yılı.
`km`	`double`	Kilometresi.
`yakitTipi`	`YakitTipi`	@Enumerated(STRING) - BENZIN, DIZEL, LPG, ELEKTRIK, HIBRIT.
`vitesTipi`	`VitesTipi`	@Enumerated - MANUEL, OTOMATIK, YARI_OTOMATIK.
`renk`	`String`	Aracın rengi.
`plakaNo`	`String`	@Column(unique = true) - Tekil indeksli plaka.
`sasiNo`	`String`	Şasi numarası.
`alisFiyati`	`double`	Galerinin alış fiyatı.
`satisFiyati`	`double`	Satış fiyatı.
`durum`	`AracDurumu`	@Enumerated - SATISTA, SATILDI, REZERVE, SERVISTE.
`eklenmeTarihi`	`LocalDateTime`	@CreationTimestamp - Otomatik tarih damgası.

Metotlar:

Metot İmzası	Açıklama
`+ getAracBilgisi(): String`	Aracın özet metnini döndürür.
`+ fiyatHesapla(): double`	*[abstract]* Polimorfik fiyat hesaplama; her alt sınıfta override edilir.
`+ aracTipiGetir(): String`	*[abstract]* Araç tipi etiketini döndürür.
`+ getter/setter'lar`	Tüm alanlar için standart erişim metotları.

3.5.1.2. Otomobil *«@Entity»*

`Arac` soyut sınıfını kalıtan somut araç tipidir. Binek otomobiller (sedan, hatchback, coupe vb.) için kullanılır. `otomobiller` tablosuna eşlenir. `fiyatHesapla()` metodunda sunroof varlığı +5% premium olarak yansıtılır.

Öznitelikler:

Alan	Tip	Açıklama
`kasaTipi`	`String`	Kasa tipi (Sedan, Hatchback, Coupe).
`kapiSayisi`	`int`	Kapı sayısı (genellikle 2 veya 4).
`sunroof`	`boolean`	Sunroof varlığı.

Metotlar:

Metot İmzası	Açıklama
`+ fiyatHesapla(): double`	Base fiyat + sunroof varsa %5 ek değer.
`+ aracTipiGetir(): String`	"Otomobil" değerini döner.

3.5.1.3. SUV *«@Entity»*

Sport Utility Vehicle segmentini temsil eden somut Entity'dir. `suvler` tablosuna eşlenir. Dört çeker ve off-road özellikleri fiyat hesaplamasına yansır.

Öznitelikler:

Alan	Tip	Açıklama
`dortCeker`	`boolean`	4x4 sistem varlığı.
`bagajHacmi`	`double`	Bagaj hacmi (litre).
`offRoad`	`boolean`	Off-road paketi varlığı.

Metotlar:

Metot İmzası	Açıklama
`+ fiyatHesapla(): double`	Base fiyat + 4x4 için %8 + off-road için %5 ek değer.
`+ aracTipiGetir(): String`	"SUV" değerini döner.

3.5.1.4. Ticari *«@Entity»*

Hafif ticari araçları (panel van, pickup, kamyonet) temsil eden somut Entity'dir. `ticariler` tablosuna eşlenir. Frigorifik özellik fiyat hesabında +12% premium yansıtılır.

Öznitelikler:

Alan	Tip	Açıklama
`tasimaKapasitesi`	`double`	Taşıma kapasitesi (kg).
`ticariTip`	`String`	Tip (Panel Van, Pickup, Kamyonet).
`frigorifik`	`boolean`	Soğutmalı gövde varlığı.

Metotlar:

Metot İmzası	Açıklama
`+ fiyatHesapla(): double`	Base fiyat + frigorifik varsa %12 ek değer.

`+ aracTipiGetir(): String`	"Ticari" değerini döner.

3.5.1.5. Motosiklet *«@Entity»*

Motosiklet segmentini temsil eden somut Entity'dir. `motosikletler` tablosuna eşlenir. 500cc üzerinde +10% premium uygulanır.

Öznitelikler:

Alan	Tip	Açıklama
`motorHacmi`	`int`	Motor hacmi (cc).
`motosikletTipi`	`String`	Tip (Sport, Cruiser, Touring, Scooter).
`ABS`	`boolean`	ABS fren sistemi varlığı.

Metotlar:

Metot İmzası	Açıklama
`+ fiyatHesapla(): double`	Base fiyat + 500cc üzerinde %10 ek değer.
`+ aracTipiGetir(): String`	"Motosiklet" değerini döner.

3.5.1.6. Kisi *«abstract @MappedSuperclass»*

Sistemdeki tüm kişi tiplerinin (Müşteri, Çalışan) ortak alanlarını tutan soyut sınıftır. `@MappedSuperclass` olarak işaretlenmiştir; kendisi bir tabloya eşlenmez. Alt sınıflar kendi tablolarında (`musteriler`, `calisanlar`) bu alanları barındırır.

Öznitelikler:

Alan	Tip	Açıklama
`ad`	`String`	Kişinin adı.
`soyad`	`String`	Kişinin soyadı.
`tcKimlik`	`String`	11 haneli TC kimlik numarası (alt sınıflarda unique).
`telefon`	`String`	Telefon numarası.
`email`	`String`	E-posta adresi.
`adres`	`String`	İkamet adresi.

Metotlar:

Metot İmzası	Açıklama
`+ getAdSoyad(): String`	Ad ve soyadı birleştirip döndürür.
`+ getter/setter'lar`	Tüm alanlar için erişim metotları.

3.5.1.7. Musteri *«@Entity»*

`Kisi`'yi kalıtan somut Entity'dir. Galeride kayıtlı müşterileri temsil eder; `musteriler` tablosuna eşlenir. `tc_kimlik` sütununda unique B-Tree indeksi vardır.

Öznitelikler:

Alan	Tip	Açıklama
`musteriId`	`String`	@Id - Birincil anahtar (UUID).
`kayitTarihi`	`LocalDateTime`	@CreationTimestamp - Sisteme kayıt tarihi.

notlar	String	VIP, pazarlık vb. serbest not alanı.

Metotlar:

Metot İmzası	Açıklama
+ getter/setter'lar	Standart erişim metotları.

3.5.1.8. Calisan *«@Entity»*

Galeride çalışan personeli temsil eden somut Entity'dir. calisanlar tablosuna eşlenir. Performans ve komisyon hesaplamalarında kullanılır.

Öznitelikler:

Alan	Tip	Açıklama
calisanId	String	@Id - Birincil anahtar.
pozisyon	String	Görev pozisyonu (Satış Temsilcisi, Yönetici vb.).
maas	double	Aylık brüt maaş.
iseBaslamaTarihi	LocalDate	İşe başlama tarihi.
toplamSatis	int	Gerçekleştirilen satış sayısı.

Metotlar:

Metot İmzası	Açıklama
+ satisYap(): void	toplamSatis sayacını bir artırır.
+ performansHesapla(): double	Satış adedine ve süreye dayalı puan döner.

3.5.1.9. Satis *«@Entity»*

Gerçekleşmiş bir satış işlemini temsil eden Entity'dir. Araç, müşteri ve çalışan ilişkileri @ManyToOne ile FK olarak tutulur. Kâr hesaplaması, satış kaydı sırasında otomatik olarak yapılır. satis_tarihi üzerinde indeks vardır (raporlama için kritik).

Öznitelikler:

Alan	Tip	Açıklama
satisId	String	@Id - Birincil anahtar.
arac	Arac	@ManyToOne(fetch = LAZY) - Satılan araç (FK).
musteri	Musteri	@ManyToOne - Alıcı müşteri (FK).
calisan	Calisan	@ManyToOne - Satışı yapan çalışan (FK).
satisFiyati	double	Satışın gerçek tutarı.
alisFiyati	double	Aracın alış fiyatı (kâr hesabı için).
kar	double	Otomatik hesaplanan kâr (satisFiyati - alisFiyati).
satisTarihi	LocalDateTime	@Column(indexed) - Satış zamanı.
odemeSekli	String	Nakit, Kredi Kartı, Finansman vb.
faturaKesildi	boolean	Fatura oluşturulduysa true.

Metotlar:

Metot İmzası	Açıklama
+ karHesapla(): double	satisFiyati - alisFiyati formülüyle kâr hesaplar.

`+ karMarji(): double`	Yüzdesel kâr marjı (kar / alisFiyati × 100).
`+ faturaOlustur(): String`	Fatura metnini formatlı şekilde üretir.

3.5.1.10. Ekspertiz *«@Entity»*

Bir araç için yapılan dijital ekspertiz raporunu temsil eder. `ekspertizler` tablosuna eşlenir. Parça-hasar eşleştirmeleri `@ElementCollection` ile ayrı bir `parca_durumlari` tablosunda tutulur.

Öznitelikler:

Alan	Tip	Açıklama
`ekspertizId`	`String`	@Id - Birincil anahtar.
`arac`	`Arac`	@ManyToOne - İncelenen araç.
`calisan`	`Calisan`	@ManyToOne - Ekspertizi yapan çalışan.
`ekspertizTarihi`	`LocalDateTime`	Ekspertizin yapıldığı zaman.
`parcaDurumlari`	`Map`	@ElementCollection - Parça adı → hasar seviyesi eşleştirmesi.
`ekspertizPuani`	`double`	Otomatik hesaplanan genel puan (0-100).
`genelDegerlendirme`	`String`	@Column(length = 1000) - Yorum metni.
`onaylandi`	`boolean`	Ekspertiz onaylandıysa true.

Metotlar:

Metot İmzası	Açıklama
`+ parcaDurumuEkle(parca, seviye)`	Parça-hasar eşleştirmesi ekler.
`+ ekspertizPuaniHesapla(): double`	Tüm parça durumlarından ortalama puan üretir.
`+ raporOlustur(): String`	Detaylı rapor metnini üretir.

3.5.1.11. SigortaBilgisi *«@Entity»*

Bir aracın sigorta poliçesi bilgilerini temsil eder. Bildirim sistemi tarafından düzenli olarak sorgulanır.

Öznitelikler:

Alan	Tip	Açıklama
`policeNo`	`String`	@Id - Poliçe numarası.
`arac`	`Arac`	@ManyToOne - Sigortalı araç.
`sigortaSirketi`	`String`	Sigorta şirketi adı.
`baslangicTarihi`	`LocalDate`	Sigorta başlangıç tarihi.
`bitisTarihi`	`LocalDate`	Sigorta bitiş tarihi.
`primTutari`	`double`	Yıllık prim tutarı.
`aktif`	`boolean`	Aktif/pasif durum.

Metotlar:

Metot İmzası	Açıklama
`+ suresiDolduMu(): boolean`	Bugün itibariyle süre dolmuş mu kontrol eder.
`+ kalanGun(): int`	Bitişe kaç gün kaldığını döner (negatif: süresi dolmuş).

3.5.1.12. MuayeneBilgisi *«@Entity»*

Bir aracın trafik muayene bilgilerini temsil eder. TÜVTÜRK muayene tarihleri ve sonuçlarını tutar.

Öznitelikler:

Alan	Tip	Açıklama
muayeneId	String	@Id - Birincil anahtar.
arac	Arac	@ManyToOne - Muayene edilen araç.
muayeneTarihi	LocalDate	Son muayene tarihi.
sonrakiMuayeneTarihi	LocalDate	Bir sonraki gerekli tarih.
gectiMi	boolean	Muayene sonucu (geçti/kaldı).
muayeneIstasyonu	String	TÜVTÜRK istasyon adı.

Metotlar:

Metot İmzası	Açıklama
+ suresiDolduMu(): boolean	Süre dolmuş mu kontrol eder.
+ kalanGun(): int	Bir sonraki muayeneye kaç gün kaldığı.

3.5.1.13. AracDurumu *«enum»*

Araç yaşam döngüsünü temsil eden tip-güvenli sabit küme: SATISTA *(satışta),* SATILDI *(satılmış),* REZERVE *(rezerve edilmiş),* SERVISTE *(serviste).*

3.5.1.14. YakitTipi *«enum»*

Desteklenen yakıt türleri: BENZIN, DIZEL, LPG, ELEKTRIK, HIBRIT.

3.5.1.15. VitesTipi *«enum»*

Desteklenen vites türleri: MANUEL, OTOMATIK, YARI_OTOMATIK.

3.5.1.16. HasarSeviyesi *«enum»*

Ekspertizdeki parça hasar kategorileri: HASARSIZ, BOYALI, DEGISEN, AGIR_HASARLI. Ekspertiz puanı hesaplamasında ağırlıklar belirler.

3.5.2. Repository Katmanı (Spring Data JPA)

Repository katmanı; Spring Data JPA framework'ünün `JpaRepository` arayüzünden türetilen yedi arayüzden oluşur. Bu arayüzler hiçbir somut implementasyon içermez; Spring Data JPA çalışma zamanında otomatik olarak proxy sınıflar üretir. Bu sayede CRUD operasyonları (`save, findById, findAll, delete`) yazılmadan kullanılabilir hale gelir.

Geliştirici, ek olarak ihtiyaç duyduğu sorguları iki yöntemle tanımlayabilir:

- **Metot ismi tabanlı sorgular:** `findByPlakaNo`, `findByMarkaIgnoreCase` gibi metotlar Spring tarafından isminden çıkarılarak otomatik üretilir.
- **Özel JPQL/SQL sorguları:** Karmaşık sorgular için `@Query` anotasyonu ile JPQL ya da native SQL yazılır.

3.5.2.1. `AracRepository` *«interface @Repository, JpaRepository»*

Araç entity'si üzerinde CRUD ve özelleştirilmiş sorgu işlemlerini yürüten Spring Data JPA arayüzüdür.

Metotlar:

Metot İmzası	Açıklama
`+ findByPlakaNo(plaka: String): Optional`	Plaka ile araç bulur (B-Tree indeksi ile O(log n) hızında).
`+ findByMarkaIgnoreCase(marka: String): List`	Marka bazlı arama (büyük/küçük harf duyarsız).
`+ findByDurum(durum: AracDurumu): List`	Belirli durumdaki araçları listeler.
`+ findBySatisFiyatiBetween(min, max): List`	Fiyat aralığı sorgusu.
`+ aramaYap(kriter: String): List`	@Query ile çoklu alanda arama (marka, model, plaka).
`+ stokDegeri(): Double`	@Query SUM(satisFiyati) — toplam stok değeri.
`+ Inherited from JpaRepository`	save, findById, findAll, deleteById, count vb.

3.5.2.2. `MusteriRepository` *«interface @Repository»*

Müşteri entity'si üzerinde sorgu yapan repository. TC kimlik üzerinde unique B-Tree indeksi sayesinde hızlı arama sunar.

Metotlar:

Metot İmzası	Açıklama
`+ findByTcKimlik(tc: String): Optional`	TC kimlik ile müşteri bulur (O(log n)).
`+ findByAdContainingIgnoreCase(ad: String): List`	Ad bazlı içerme sorgusu.
`+ existsByTcKimlik(tc: String): boolean`	TC kimlik kayıtlı mı kontrol eder.
`+ findAllByOrderByKayitTarihiDesc(): List`	En yeni müşteriler önce.

3.5.2.3. `SatisRepository` *«interface @Repository»*

Satış entity'si üzerinde sorgu ve agregasyon işlemlerini yapar. Raporlama için kritik metotlar içerir.

Metotlar:

Metot İmzası	Açıklama
`+ findAllByOrderBySatisTarihiDesc(): List`	En yeni satışlar önce.
`+ findByMusteri_TcKimlik(tc): List`	Bir müşterinin tüm satışları.
`+ findByCalisan_CalisanId(id): List`	Bir çalışanın tüm satışları.
`+ findBySatisTarihiBetween(t1, t2): List`	Tarih aralığında satışlar.
`+ aylikSatislar(yil: int): List`	@Query GROUP BY ay; aylık satış adedi ve toplamı.
`+ toplamCiro(): Double`	@Query SUM(satisFiyati).
`+ toplamKar(): Double`	@Query SUM(kar).
`+ markayaGoreCiro(): List`	@Query GROUP BY marka; raporlama için.

3.5.2.4. `EkspertizRepository` *«interface @Repository»*

Ekspertiz raporları için repository. Bir aracın en son ekspertizine tek sorgu ile erişim sağlar.

Metotlar:

Metot İmzası	Açıklama
`+ findByArac_AracIdOrderByEkspertizTarihiDesc(id): List`	Aracın tüm ekspertizleri, en yeniden eskiye.
`+ findFirstByArac_AracIdOrderByEkspertizTarihiDesc(id): Optional`	Aracın en son ekspertizi (LIMIT 1).
`+ findByCalisan_CalisanId(id): List`	Bir çalışanın yaptığı ekspertizler.
`+ ortalamaPuan(): Double`	@Query AVG(ekspertizPuani).

3.5.2.5. `CalisanRepository` *«interface @Repository»*

Metotlar:

Metot İmzası	Açıklama
`+ findByPozisyon(pozisyon: String): List`	Pozisyon bazlı listele.
`+ findAllByOrderByToplamSatisDesc(): List`	En çok satış yapan çalışan önce.

3.5.2.6. `SigortaRepository` *«interface @Repository»*

Metotlar:

Metot İmzası	Açıklama
`+ findByArac_AracId(id): List`	Bir aracın tüm sigortaları.
`+ findByBitisTarihiBefore(date): List`	Süresi dolmuş sigortalar.
`+ findByBitisTarihiBetween(t1, t2): List`	Yaklaşan süre sonu sigortaları.

3.5.2.7. `MuayeneRepository` *«interface @Repository»*

Metotlar:

Metot İmzası	Açıklama
`+ findByArac_AracId(id): List`	Bir aracın tüm muayeneleri.
`+ findBySonrakiMuayeneTarihiBefore(date): List`	Süresi dolmuş muayeneler.
`+ findBySonrakiMuayeneTarihiBetween(t1, t2): List`	Yaklaşan muayeneler.

3.5.3. Service Katmanı (İş Mantığı)

Service katmanı; Controller ile Repository arasında köprü görevi üstlenir ve tüm iş mantığını barındırır. Tüm Service sınıfları `@Service` anotasyonu ile işaretlenir; Spring tarafından otomatik olarak singleton bean olarak yönetilir. Bağımlılıklar (Repository'ler) constructor injection ile alınır. Birden fazla veritabanı işlemi yapan kritik metotlar `@Transactional` ile sarmalanır.

3.5.3.1. `AracService` *«@Service»*

Araç ile ilgili iş kurallarını yürüten servistir. Plaka tekilliği kontrolü, satılmış araçların silinememesi gibi kuralları uygular.

Öznitelikler:

Alan	Tip	Açıklama
`aracRepository`	`AracRepository`	Constructor injection ile.

Metotlar:

Metot İmzası	Açıklama
`+ aracEkle(arac: Arac): Arac`	Plaka tekilliği kontrolü sonrası kaydeder.
`+ aracSil(id: String): void`	SATILDI durumunda silmez; IsKuraliException atar.
`+ aracGuncelle(id, yeniArac): Arac`	Mevcut araç kaydını günceller.
`+ aracBul(id: String): Arac`	Bulunamazsa IsKuraliException atar.
`+ tumAraclar(): List`	Tüm araçları listeler.
`+ ara(kriter: String): List`	Çoklu alanda arama yapar.
`+ filtrele(durum, fiyatMin, fiyatMax, marka): List`	JPA Specification ile dinamik filtreleme.
`+ stokDegeri(): double`	Mevcut stoğun toplam satış değeri.
`+ markaIstatistikleri(): Map`	Marka bazında araç sayıları.

3.5.3.2. `MusteriService` *«@Service»*

Müşteri yönetimini yürüten servistir. TC kimlik tekilliği ve format doğrulamasını yapar.

Öznitelikler:

Alan	Tip	Açıklama
`musteriRepository`	`MusteriRepository`	Constructor injection ile.

Metotlar:

Metot İmzası	Açıklama
`+ musteriEkle(musteri: Musteri): Musteri`	TC tekilliği kontrolü sonrası kaydeder.
`+ musteriSil(id: String): void`	Müşteri kaydını siler.
`+ musteriGuncelle(id, yeniBilgi): Musteri`	Müşteri bilgilerini günceller.
`+ tcKimlikleAra(tc: String): Musteri`	TC ile müşteri bulur.
`+ adaGoreAra(ad: String): List`	Ad bazında içerme araması.
`+ tumMusteriler(): List`	Tüm müşterileri listeler.

`+ musteriSayisi(): long`	Toplam müşteri sayısı.

3.5.3.3. SatisService *«@Service @Transactional»*

Satış işlemlerini yürüten en kritik servistir. Tüm metotları `@Transactional` ile sarmalanmıştır. `satisYap()` metodu; aracı bulup durumunu günceller, satış kaydı oluşturur ve müşteri-çalışan ilişkilerini kurar — tümü tek bir atomik veritabanı işlemi içinde yürütülür.

Öznitelikler:

Alan	Tip	Açıklama
`satisRepository`	`SatisRepository`	Constructor injection ile.
`aracRepository`	`AracRepository`	Constructor injection ile.
`musteriRepository`	`MusteriRepository`	Constructor injection ile.
`calisanRepository`	`CalisanRepository`	Constructor injection ile.

Metotlar:

Metot İmzası	Açıklama
`+ satisYap(istek: SatisIstegi): Satis`	Atomik satış işlemi: araç bul, müşteri bul, çalışan bul, kâr hesapla, satış kaydet, araç durumunu SATILDI yap.
`+ satisIptal(satisId: String): void`	24 saat içindeyse iptal eder; aracı tekrar SATISTA yapar.
`+ faturaUret(satisId: String): String`	Satış için formatlı fatura metni üretir.
`+ satisBul(id: String): Satis`	Bulunamazsa IsKuraliException atar.
`+ tumSatislar(): List`	Tüm satışları en yeniden eskiye listeler.
`+ musteriSatislari(tc: String): List`	Bir müşterinin satış geçmişi.
`+ calisanSatislari(id: String): List`	Bir çalışanın satışları.
`+ tarihAraligindaSatislar(t1, t2): List`	Belirli tarih aralığındaki satışlar.

3.5.3.4. EkspertizService *«@Service»*

Öznitelikler:

Alan	Tip	Açıklama
`ekspertizRepository`	`EkspertizRepository`	Constructor injection ile.
`aracRepository`	`AracRepository`	Constructor injection ile.

Metotlar:

Metot İmzası	Açıklama
`+ ekspertizEkle(istek: EkspertizIstegi): Ekspertiz`	Yeni ekspertiz oluşturur, parça durumlarını kaydeder, puan hesaplar.
`+ aracEkspertizGecmisi(aracId: String): List`	Bir aracın tüm ekspertiz geçmişi.
`+ aracSonEkspertizi(aracId: String): Ekspertiz`	Aracın en son ekspertiz raporu.

`+ tumEkspertizler(): List`	Tüm ekspertizleri listeler.

3.5.3.5. BildirimService *«@Service»*

Sigorta ve muayene tarihlerinin takibini yapar. Yaklaşan ve süresi dolmuş kayıtlar için Dashboard'a uyarı üretir.

Öznitelikler:

Alan	Tip	Açıklama
`sigortaRepository`	`SigortaRepository`	Constructor injection ile.
`muayeneRepository`	`MuayeneRepository`	Constructor injection ile.

Metotlar:

Metot İmzası	Açıklama
`+ yaklasanSigortalar(gun: int): List`	Belirli gün sayısı içinde süresi dolacak sigortalar.
`+ yaklasanMuayeneler(gun: int): List`	Belirli gün sayısı içinde süresi dolacak muayeneler.
`+ suresiDolmusSigortalar(): List`	Süresi geçmiş sigortalar.
`+ suresiDolmusMuayeneler(): List`	Süresi geçmiş muayeneler.
`+ tumBildirimler(): List`	Dashboard için tüm uyarıların özet listesi.

3.5.3.6. RaporlayiciService *«@Service»*

İş zekâsı raporlarını üreten servistir. Raporlama metrikleri (toplam ciro, kâr, en çok satılan marka) doğrudan SQL agregasyonları ile veritabanı tarafında hesaplanır; uygulama sunucusuna gereksiz veri taşınmaz.

Öznitelikler:

Alan	Tip	Açıklama
`satisRepository`	`SatisRepository`	Constructor injection ile.
`aracRepository`	`AracRepository`	Constructor injection ile.
`musteriRepository`	`MusteriRepository`	Constructor injection ile.
`calisanRepository`	`CalisanRepository`	Constructor injection ile.

Metotlar:

Metot İmzası	Açıklama
`+ dashboardOzet(): DashboardDTO`	Dashboard kartları için tüm metrikler (araç sayısı, satış adedi, ciro, kâr).
`+ aylikSatisRaporu(yil: int): List`	12 ay için satış adedi ve ciro grafiği verisi.
`+ markaSatisAnalizi(): List`	En çok satılan markalar; Recharts pie chart için.
`+ calisanPerformansRaporu(): List`	Çalışan başına satış adedi ve toplam ciro.
`+ stokDegerRaporu(): StokDegerDTO`	Mevcut stoğun toplam değeri ve marka dağılımı.
`+ karMarjAnalizi(): KarMarjDTO`	Genel kâr marjı, ortalama satış fiyatı, kar oranı.

3.5.4. Controller Katmanı (REST API)

Controller katmanı; gelen HTTP isteklerini karşılayan ve uygun Service metoduna yönlendiren altı sınıftan oluşur. Tüm Controller'lar `@RestController` ile işaretlenir ve `@RequestMapping` ile temel URL yollarını alırlar. Service bağımlılıkları constructor injection ile enjekte edilir; iş mantığı içermezler.

3.5.4.1. `AracController` *«@RestController, @RequestMapping("/api/araclar")»*

Öznitelikler:

Alan	Tip	Açıklama
`aracService`	`AracService`	Constructor injection ile.

Metotlar:

Metot İmzası	Açıklama
`@GetMapping → tumAraclar(): List`	GET /api/araclar
`@GetMapping("/{id}") → aracBul(id): Arac`	GET /api/araclar/{id}
`@PostMapping → aracEkle(@RequestBody arac): Arac`	POST /api/araclar
`@PutMapping("/{id}") → aracGuncelle(id, @RequestBody): Arac`	PUT /api/araclar/{id}
`@DeleteMapping("/{id}") → aracSil(id): void`	DELETE /api/araclar/{id}
`@GetMapping("/ara") → ara(@RequestParam q): List`	GET /api/araclar/ara?q=...
`@GetMapping("/filtrele") → filtrele(...): List`	Çoklu kriterli filtreleme.
`@GetMapping("/stok-degeri") → stokDegeri(): double`	GET /api/araclar/stok-degeri

3.5.4.2. `MusteriController` *«@RestController, @RequestMapping("/api/musteriler")»*

Öznitelikler:

Alan	Tip	Açıklama
`musteriService`	`MusteriService`	Constructor injection ile.

Metotlar:

Metot İmzası	Açıklama
`@GetMapping → tumMusteriler(): List`	GET /api/musteriler
`@GetMapping("/{tc}") → tcIleAra(tc): Musteri`	GET /api/musteriler/{tc}
`@PostMapping → musteriEkle(@RequestBody): Musteri`	POST /api/musteriler
`@PutMapping("/{id}") → guncelle(id, @RequestBody): Musteri`	PUT /api/musteriler/{id}
`@DeleteMapping("/{id}") → musteriSil(id)`	DELETE /api/musteriler/{id}
`@GetMapping("/ara") → adaGoreAra(@RequestParam ad): List`	GET /api/musteriler/ara?ad=...

3.5.4.3. SatisController *«@RestController, @RequestMapping("/api/satislar")»*

Öznitelikler:

Alan	Tip	Açıklama
satisService	SatisService	Constructor injection ile.

Metotlar:

Metot İmzası	Açıklama
@GetMapping → tumSatislar(): List	GET /api/satislar
@GetMapping("/{id}") → satisBul(id): Satis	GET /api/satislar/{id}
@PostMapping → satisYap(@RequestBody istek): Satis	POST /api/satislar
@PostMapping("/{id}/iptal") → iptalEt(id): void	POST /api/satislar/{id}/iptal
@GetMapping("/{id}/fatura") → fatura(id): String	GET /api/satislar/{id}/fatura
@GetMapping("/musteri/{tc}") → musteriSatislari(tc): List	Müşteri satış geçmişi.
@GetMapping("/calisan/{id}") → calisanSatislari(id): List	Çalışan satış listesi.

3.5.4.4. EkspertizController *«@RestController, @RequestMapping("/api/ekspertizler")»*

Öznitelikler:

Alan	Tip	Açıklama
ekspertizService	EkspertizService	Constructor injection ile.

Metotlar:

Metot İmzası	Açıklama
@GetMapping → tumEkspertizler()	GET /api/ekspertizler
@PostMapping → ekspertizEkle(@RequestBody)	POST /api/ekspertizler
@GetMapping("/arac/{aracId}") → aracEkspertizleri(aracId)	Bir aracın ekspertiz geçmişi.
@GetMapping("/arac/{aracId}/son") → sonEkspertiz(aracId)	En son ekspertiz raporu.

3.5.4.5. BildirimController *«@RestController, @RequestMapping("/api/bildirimler")»*

Öznitelikler:

Alan	Tip	Açıklama
bildirimService	BildirimService	Constructor injection ile.

Metotlar:

Metot İmzası	Açıklama
`@GetMapping("/sigorta/yaklasan/{gun}") → yaklasanSigortalar(gun)`	Belirtilen gün sayısı için yaklaşan sigortalar.
`@GetMapping("/muayene/yaklasan/{gun}") → yaklasanMuayeneler(gun)`	Yaklaşan muayeneler.
`@GetMapping("/sigorta/dolmus") → suresiDolmusSigortalar()`	Süresi geçmiş sigortalar.
`@GetMapping("/muayene/dolmus") → suresiDolmusMuayeneler()`	Süresi geçmiş muayeneler.
`@GetMapping → tumBildirimler()`	Dashboard için özet liste.

3.5.4.6. `RaporController` *«@RestController, @RequestMapping("/api/raporlar")»*

Öznitelikler:

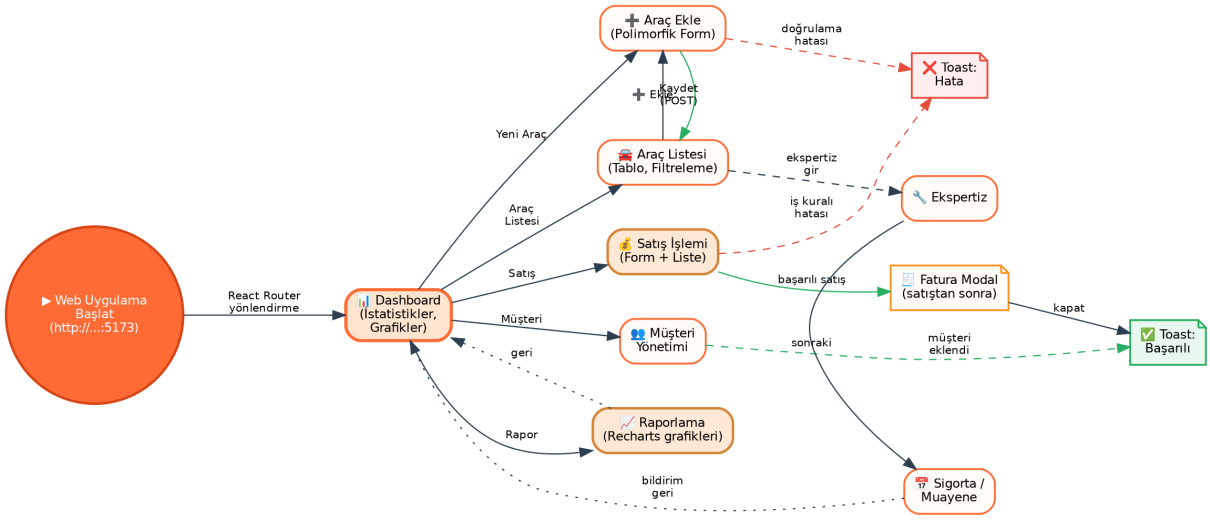
Alan	Tip	Açıklama
`raporlayiciService`	`RaporlayiciService`	Constructor injection ile.

Metotlar:

Metot İmzası	Açıklama
`@GetMapping("/dashboard") → dashboardOzet()`	Dashboard kartları için tüm metrikler.
`@GetMapping("/aylik/{yil}") → aylikSatis(yil)`	12 aylık satış grafiği verisi.
`@GetMapping("/marka") → markaSatisAnalizi()`	Marka bazlı satış dağılımı.
`@GetMapping("/calisan-performans") → calisanPerformans()`	Çalışan satış performansı.
`@GetMapping("/stok-deger") → stokDeger()`	Stok değeri raporu.
`@GetMapping("/kar-marj") → karMarj()`	Kâr marjı analizi.

3.5.5. Sunum Katmanı (React Bileşenleri)

Frontend; React 18 ile geliştirilmiş bileşen tabanlı bir tek-sayfa uygulamasıdır. Üç ana bileşen kategorisinden oluşur: **sayfa bileşenleri**, **düzen (layout) bileşenleri** ve **yeniden kullanılabilir UI bileşenleri**.



Şekil 3.5: *AOGS UI Navigasyon Akış Diyagramı — React Router ile yönetilen 8 sayfa arasındaki geçişler ve bildirim/modal etkileşimleri.*

Sayfa Bileşenleri

- `Dashboard.jsx`**:** Dört istatistik kartı (Toplam Araç, Aylık Satış, Aylık Ciro, Aktif Müşteri), Recharts ile aylık satış grafiği, son satışlar tablosu, yaklaşan bildirim özeti.
- `AracListesi.jsx`**:** Tüm araçların tablosu; marka, model, plaka, durum bazlı filtreleme; her satırda düzenleme/silme aksiyonları.
- `AracEkle.jsx`**:** Polimorfik araç ekleme formu; araç tipi seçimine göre dinamik alanlar (sunroof, dortCeker, frigorifik, motorHacmi vb.).
- `MusteriYonetimi.jsx`**:** Müşteri kayıt formu, TC ile arama, müşteri listesi.
- `SatisIslemi.jsx`**:** Yeni satış formu (araç, müşteri, ödeme), satış geçmişi tablosu, fatura modal'ı.
- `Ekspertiz.jsx`**:** Ekspertiz raporu oluşturma; parça bazlı hasar seviyesi seçimi.
- `SigortaMuayene.jsx`**:** Sigorta/muayene kayıtları, yaklaşan ve süresi dolmuş listeler, renk kodlu badge'ler.
- `Raporlama.jsx`**:** Çoklu rapor türü; Recharts ile bar, pie ve line grafikleri.

Düzen Bileşenleri

- `Sidebar.jsx`**:** Sol kalıcı navigasyon menüsü; 8 sayfanın linki, aktif menü vurgusu (turuncu arka plan).
- `TopBar.jsx`**:** Üst başlık çubuğu; uygulama adı, kullanıcı bilgisi, tema (gelecek).

Yeniden Kullanılabilir UI Bileşenleri

- **Kart:** Beyaz arka plan, soft turuncu kenar, yumuşak gölge (`rounded-xl shadow-soft`).
- **VurguluButon:** Birincil eylemler için turuncu buton; hover'da rengi koyulaşır.
- **IkincilButon:** Daha az önemli eylemler için outline stilli buton.

- **Input, TextArea, Select:** Tutarlı stilde form alanları; doğrulama hatasında kırmızı border.
- **Modal:** Fatura, onay diyalogları için arka planı blur'lu modal pencere.
- **Toast:** Başarı (yeşil) ve hata (kırmızı) bildirimleri için geçici popup.

API İstemcisi (api/client.js)

Tüm REST çağrıları tek bir merkezi modülde yönetilir. axios üzerine kurulu olan bu istemci; her endpoint için tip-güvenli fonksiyonlar sunar (`aracApi.liste()`, `satisApi.yap(istek)` vb.). Ortak konfigürasyonlar (base URL, header'lar, error interceptor) tek yerde tanımlanır.

3.6. Tasarım Desenleri (Design Patterns)

AOGS tasarımında modern web mimarisi ve Spring framework'ünün endüstri standardı desenleri uygulanmıştır. Aşağıdaki dört desen öne çıkmaktadır: **Dependency Injection**, **Repository Pattern**, **DTO Pattern** ve **Strategy Pattern**. Her desenin seçilme gerekçesi, AOGS'taki uygulama alanı ve sağladığı fayda aşağıda detaylı olarak anlatılmaktadır.

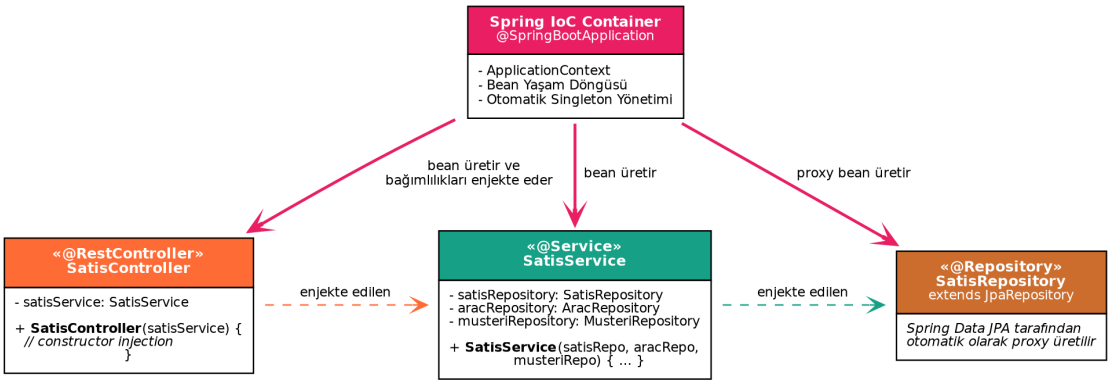
Desen	Kategori	AOGS'taki Kullanım Yeri
Dependency Injection	Yapısal	Spring IoC Container; tüm Service ve Controller bağımlılıkları
Repository Pattern	Yapısal (Veri Erişim)	Spring Data JPA Repository'leri; CRUD ve özel sorgular
DTO Pattern	Yapısal	İstemci-Sunucu arasında veri taşıma; SatisIstegi, EkspertizIstegi vb.
Strategy Pattern	Davranışsal	JPA Specification API ile dinamik filtreleme

Tablo 3.5: *AOGS'ta Uygulanan Tasarım Desenleri Özeti*

3.6.1. Dependency Injection (Bağımlılık Enjeksiyonu)

Amaç: Bir sınıfın ihtiyaç duyduğu nesnelerin (bağımlılıkların) bizzat o sınıf tarafından oluşturulması yerine; dışarıdan bir kontrolör (container) tarafından sağlanmasını mümkün kılmak. Bu sayede sınıflar arası gevşek bağlılık, kolay test edilebilirlik ve yapılandırma esnekliği sağlanır.

AOGS'taki Gerekçe: Sistemde 6 Service, 7 Repository ve 6 Controller bulunur. Her Service'in birden fazla Repository'ye, her Controller'ın bir Service'e ihtiyacı vardır. Bu bağımlılıkları manuel olarak `new` anahtar kelimesiyle oluşturmak; bağlılık karmaşası, test zorluğu ve yapılandırma tekrarına yol açardı. Spring'in IoC Container'ı bu sorunu otomatik olarak çözer: bean'leri keşfeder, yaşam döngülerini yönetir ve constructor parametrelerine bakarak uygun bağımlılıkları enjekte eder.



Şekil 3.7: *Dependency Injection — Spring IoC Container tüm bean'leri yönetir; Controller, Service ve Repository'ler arasındaki bağlantıyı otomatik olarak kurar.*

AOGS'ta Uygulanma Şekli:

- Her sınıf uygun bir Spring stereotype anotasyonu ile işaretlenir: `@RestController`, `@Service`, `@Repository`.

- Bağımlılıklar **final** alan olarak tanımlanır ve **constructor injection** ile alınır (Lombok ile `@RequiredArgsConstructor` kullanılabilir).
- Spring uygulama başlatıldığında ApplicationContext'i kurar; tüm `@Component` sınıflarını tarar; bean grafiğini oluşturur ve gerekli enjeksiyonları otomatik yapar.
- Her bean varsayılan olarak **singleton** scope'undadır; tüm uygulama yaşam döngüsü boyunca tek örnek kullanılır.

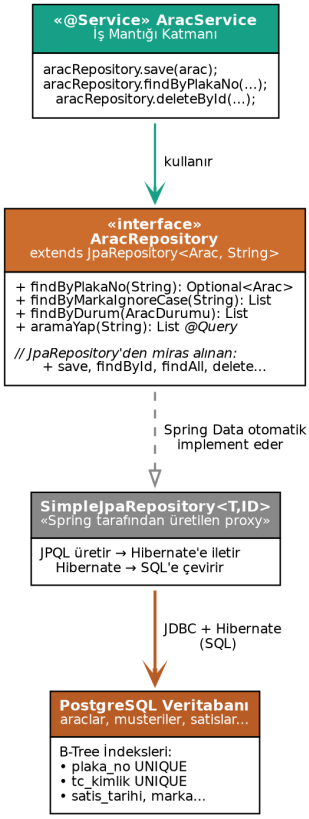
Sağlanan Faydalar:

- **Test edilebilirlik:** Constructor parametreleri sayesinde Service sınıfları, mock Repository'ler ile birim testlerde kolayca test edilebilir.
- **Gevşek bağlılık:** Sınıflar somut implementasyonlara değil, arayüzlere (Repository) bağlıdır; implementasyon değişiklikleri sınıfı etkilemez.
- **Yaşam döngüsü yönetimi:** Bean'lerin oluşturulması, yapılandırılması ve yok edilmesi merkezi olarak yapılır.
- **Aspect-Oriented destek:** `@Transactional`, `@Cacheable` gibi cross-cutting concern'ler bean proxy'leri sayesinde transparan biçimde uygulanır.

3.6.2. Repository Pattern

Amaç: İş mantığı katmanını veri erişim mekaniğinden soyutlamak. Service sınıfları SQL, JDBC veya ORM detayları ile uğraşmadan; basit nesne odaklı bir arayüz üzerinden veriye erişir.

AOGS'taki Gerekçe: Service'ler iş mantığına odaklanmalıdır; veritabanı sorgularını yazmak, JDBC bağlantısı yönetmek, ResultSet'i nesneye çevirmek gibi düşük seviyeli işlerle uğraşmamalıdır. Spring Data JPA'nın `JpaRepository` arayüzü, Repository deseninin endüstri standardı bir uygulamasıdır. Geliştirici sadece arayüzü tanımlar; Spring çalışma zamanında otomatik olarak proxy implementasyonu üretir.



Şekil 3.8: *Repository Pattern — Service katmanından veritabanına kadar olan akış; Spring'in otomatik üretilen proxy'si JPQL ve SQL üretimini yönetir.*

AOGS'ta Uygulanma Şekli:

- Her Entity için bir Repository arayüzü tanımlanır (`AracRepository`, `SatisRepository` vb.).
- Arayüz `JpaRepository<Entity, ID>`'den türer; otomatik olarak save, findById, findAll, deleteById, count gibi metotları kazanır.
- Özel sorgular için iki yöntem mevcuttur: **(1)** Metot ismi tabanlı sorgular (`findByPlakaNo`) — Spring metot imzasından otomatik JPQL üretir. **(2)** `@Query` anotasyonlu metotlar — JPQL ya da native SQL açıkça yazılır.
- Spring çalışma zamanında `SimpleJpaRepository` tabanlı bir proxy üretir; bu proxy çağrıları Hibernate'e iletir; Hibernate veritabanına SQL gönderir.

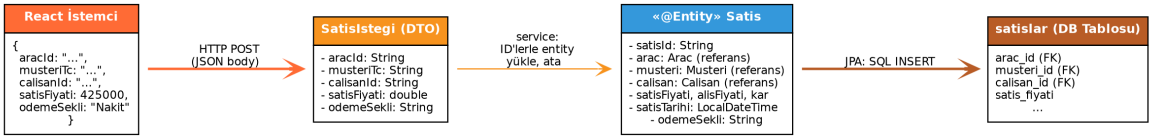
Sağlanan Faydalar:

- **Soyutlama:** Service sınıfları SQL bilmeden çalışır; veritabanı şeması Repository arayüzünün arkasında saklanır.
- **Verimlilik:** CRUD operasyonları yazılmadan kullanılabilir; geliştirici sadece domain'e özgü sorgulara odaklanır.
- **Veritabanı bağımsızlığı:** Aynı Repository kodu PostgreSQL, MySQL, H2 gibi farklı veritabanlarında çalışır.
- **Test edilebilirlik:** Repository'ler arayüz olduğundan kolayca mock'lanabilir.

3.6.3. DTO (Data Transfer Object) Pattern

Amaç: Sistem sınırları (örneğin istemci-sunucu) arasında veri taşıyan nesneleri, iç domain modellerinden ayırmak. Bu sayede API sözleşmesi domain modelinin değişikliklerinden etkilenmez ve iç model dış dünyaya sızmaz.

AOGS'taki Gerekçe: Frontend ile backend arasındaki iletişim JSON üzerinden gerçekleşir. Eğer JPA Entity'leri doğrudan API yanıtı olarak verilseydi şu sorunlar oluşurdu: **(1)** Lazy loading sırasında sonsuz döngü (satış → araç → satışlar → ...). **(2)** İç model değişikliklerinin API'ye sızması; istemcinin kırılması. **(3)** Frontend'in ihtiyaç duymadığı alanların gönderilmesi (performans/güvenlik kaybı). DTO'lar bu sorunları ortadan kaldırır.



Şekil 3.9: DTO Pattern — React istemcisinden gelen JSON verisi, DTO sınıfına dönüştürülür; Service katmanı DTO'dan Entity'yi üretir ve veritabanına kaydeder.

AOGS'ta Uygulanma Şekli:

- Karmaşık iş akışları için özel DTO sınıfları tanımlanır: `SatisIstegi`, `EkspertizIstegi`, `DashboardDTO`, `AylikRaporDTO` vb.
- Controller'lar gelen istekleri `@RequestBody` ile DTO olarak alır.
- Service katmanı, DTO'daki ID'leri kullanarak Repository üzerinden ilgili Entity'leri yükler ve yeni Entity nesnesini oluşturur.
- Yanıt olarak da gerektiğinde Entity yerine DTO döndürülür (özellikle raporlama metriklerinde).
- Jackson kütüphanesi (Spring Boot ile gelir) JSON ↔ Java nesnesi dönüşümünü otomatik yapar.

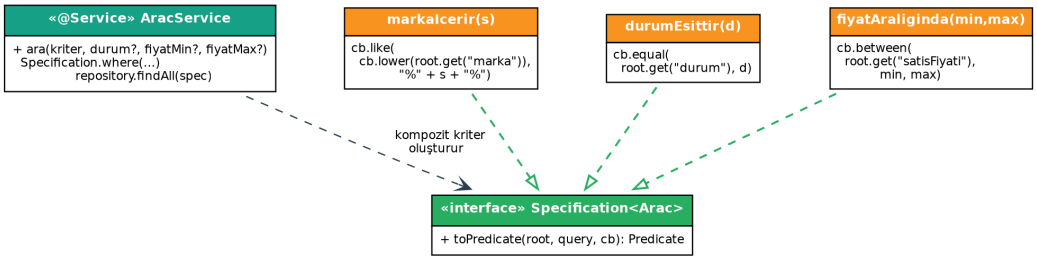
Sağlanan Faydalar:

- **API kararlılığı:** Domain modeli değişse bile API sözleşmesi korunabilir.
- **Performans:** Sadece gerekli alanlar gönderilir; ağ trafiği azalır.
- **Güvenlik:** Hassas alanlar (örn: şifre hash'i) yanlışlıkla API'ye sızmaz.
- **İstemci konforu:** DTO'lar istemcinin tam ihtiyacına göre şekillendirilebilir; gereksiz iç içe ilişkiler düzleştirilebilir.

3.6.4. Strategy Pattern (JPA Specification ile)

Amaç: Bir algoritma ailesini tanımlamak, her birini kapsüllemek ve çalışma zamanında aralarında değiştirilebilir kılmak.

AOGS'taki Gerekçe: Araç listesi, kullanıcının seçimine göre farklı kriterlerle filtrelenebilmelidir: marka, durum, fiyat aralığı, yıl. Bu kriterler tek tek de, kombinasyonlar halinde de uygulanabilir. Her kriter kombinasyonu için ayrı bir Repository metodu yazmak ('findByMarka', 'findByMarkaAndDurum', 'findByMarkaAndDurumAndFiyat...') sürdürülemez bir hâl alırdı. Spring Data JPA'nın **Specification API**'si Strategy deseninin modern bir uygulamasıdır: her kriter bir `Specification<Arac>` nesnesi olarak tanımlanır ve bunlar `and()`, `or()` operatörleri ile çalışma zamanında birleştirilir.



Şekil 3.10: *Strategy Pattern —* `AracService` *(Context); farklı* `Specification` *stratejileri (markaIcerir, durumEsittir, fiyatAraliginda) çalışma zamanında birleştirilir.*

AOGS'ta Uygulanma Şekli:

- `AracRepository` ek olarak `JpaSpecificationExecutor<Arac>` arayüzünü implemente eder.
- Statik bir `AracSpecifications` sınıfı yaygın kriterleri ayrı metotlar olarak tanımlar: `markaIcerir(s)`, `durumEsittir(d)`, `fiyatAraliginda(min, max)`.
- `AracService.filtrele()` metodu, kullanıcının seçtiği kriterlere göre bu Specification'ları `Specification.where(...).and(...)` ile birleştirir.
- Spring; bu kompozit Specification'dan otomatik olarak optimum bir JPQL/SQL sorgusu üretir; veritabanı tarafında B-Tree indeksleri kullanılarak hızlıca çalıştırılır.

Sağlanan Faydalar:

- **Açık-Kapalı İlkesi:** Yeni bir filtre kriteri eklemek, mevcut kodu değiştirmeden mümkündür; sadece yeni bir Specification metodu eklenir.
- **Kompozisyon:** Kriterler `and()` ve `or()` ile çalışma zamanında birleştirilebilir; eksponansiyel sayıda metot yazma ihtiyacı ortadan kalkar.
- **Tek Sorumluluk:** Her Specification kendi kriterinden sorumludur.
- **SQL üretimi:** Her şey JPQL üzerinden çalıştığı için veritabanı performansından (indeksler, query planner) yararlanır.

SÖZLÜK (GLOSSARY)

Bu sözlük, AOGS tasarım raporunda sıkça kullanılan teknik terimleri ve proje-özel kısaltmaları Türkçe açıklamalarıyla içermektedir.

Terim	Tanım
AOGS	Akıllı Oto Galeri Sistemi.
API	Application Programming Interface (Uygulama Programlama Arayüzü). Bir yazılımın başka bir yazılımla iletişim kurmasını sağlayan kurallar bütünü.
axios	JavaScript için Promise tabanlı HTTP istemci kütüphanesi. Frontend'den backend'e REST istekleri yapmak için kullanılır.
B-Tree	Balanced tree (dengeli ağaç). PostgreSQL'de varsayılan indeks yapısı; arama, ekleme, silme O(log n) karmaşıklığında çalışır.
Bean	Spring'de IoC Container tarafından yönetilen nesne. @Component, @Service, @Repository, @RestController ile işaretlenir.
CORS	Cross-Origin Resource Sharing. Bir tarayıcının farklı bir domain'deki API'yi çağırmasına izin veren güvenlik politikası.
CRUD	Create, Read, Update, Delete - Temel veri işlemleri.
Dependency Injection	Bağımlılık Enjeksiyonu. Bir sınıfın ihtiyaç duyduğu nesnelerin dışarıdan sağlanması.
DTO	Data Transfer Object. Sistem sınırları arasında veri taşıyan basit nesne.
Endpoint	REST API'de belirli bir URL'ye karşılık gelen işlev (örn: GET /api/araclar).
Entity	JPA'da bir veritabanı tablosuna eşlenmiş sınıf. @Entity anotasyonu ile işaretlenir.
FK (Foreign Key)	Yabancı anahtar. Bir tablodaki sütunun başka bir tablonun primary key'ine referans vermesi.
Hibernate	En yaygın JPA implementasyonu. Java nesneleri ile veritabanı tabloları arasında ORM eşlemesi yapar.
HTTP	HyperText Transfer Protocol. Web üzerinde veri iletim protokolü.
IoC	Inversion of Control (Kontrolün Tersine Çevrilmesi). Spring'in bean yönetim modelinin temeli.
JDBC	Java Database Connectivity. Java'da veritabanı bağlantısı için standart API.
JPA	Java Persistence API. Java'da ORM için standart spesifikasyon.

JPQL	Java Persistence Query Language. JPA'nın sorgu dili; SQL'e benzer ama Java sınıfları üzerinde çalışır.
JSON	JavaScript Object Notation. Web'de yaygın kullanılan, insan tarafından okunabilir veri formatı.
JVM	Java Virtual Machine. Java bytecode'larını çalıştıran sanal makine.
LTS	Long-Term Support. Uzun süreli destek alan yazılım sürümü (Java 17 LTS gibi).
OCP	Open-Closed Principle. Açık-Kapalı İlkesi; yazılım varlıklarının genişlemeye açık ama değişime kapalı olması.
OOP	Object-Oriented Programming (Nesne Yönelimli Programlama).
ORM	Object-Relational Mapping. Nesne yönelimli kodu ilişkisel veritabanına eşleyen teknik.
PK	Primary Key (Birincil Anahtar). Tablodaki her satırı benzersiz tanımlayan sütun.
PostgreSQL	Açık kaynaklı, güçlü, ACID uyumlu ilişkisel veritabanı yönetim sistemi.
React	Facebook tarafından geliştirilen, bileşen bazlı modern frontend kütüphanesi.
Repository	Veri erişim katmanı için tasarım deseni. Spring Data JPA'da JpaRepository arayüzü ile uygulanır.
REST	Representational State Transfer. HTTP üzerinden kaynak odaklı API tasarım stili.
SOLID	Beş nesne yönelimli tasarım ilkesinin kısaltması.
SPA	Single Page Application. Tek HTML sayfası üzerinde JavaScript ile dinamik olarak içerik değiştiren web uygulaması.
Spring Boot	Spring framework'ünün kolayca başlatılabilen, opinionated dağıtımı.
Spring Data JPA	Repository pattern'i otomatize eden Spring modülü.
SQL	Structured Query Language. İlişkisel veritabanı sorgu dili.
SRS	Software Requirements Specification. Yazılım Gereksinimleri Spesifikasyonu.
Strategy Pattern	Bir algoritma ailesini tanımlayarak çalışma zamanında değiştirilebilmesini sağlayan davranışsal desen.
Supabase	Açık kaynaklı, Firebase alternatifi olan yönetilen PostgreSQL platformu.
Tailwind CSS	Utility-first yaklaşımıyla CSS yazımını basitleştiren modern stil framework'ü.

TLS/SSL	Transport Layer Security. Ağ üzerinde şifreli iletişim sağlayan protokol.
Trade-off	Ödünleşim. Birbirine zıt iki kalite hedefi arasında yapılan bilinçli tercih.
Transactional	@Transactional anotasyonu ile sarmalanan metot, atomik bir veritabanı işlemi içinde çalışır.
UML	Unified Modeling Language. Yazılım sistemlerini görselleştirmek için kullanılan standart modelleme dili.
Use Case	Kullanım Durumu. Bir aktörün sistemde gerçekleştirebileceği iş akışı.
UUID	Universally Unique Identifier. Evrensel benzersiz tanımlayıcı.
Vite	Modern, hızlı bir frontend build aracı; React projelerinin geliştirme ve üretim derlemesini sağlar.

REFERANSLAR

AOGS tasarım raporunun hazırlanması ve tasarım kararlarının şekillenmesi sürecinde aşağıdaki kaynaklardan yararlanılmıştır. Referanslar IEEE alıntı formatında verilmiştir.

[1] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA, USA: Addison-Wesley, 1994.

[2] B. Bruegge and A. H. Dutoit, *Object-Oriented Software Engineering Using UML, Patterns, and Java*, 3rd ed. Upper Saddle River, NJ, USA: Prentice Hall, 2010.

[3] R. C. Martin, *Clean Code: A Handbook of Agile Software Craftsmanship*. Upper Saddle River, NJ, USA: Prentice Hall, 2008.

[4] R. Johnson et al., "Spring Framework Reference Documentation." [Çevrimiçi]. Erişim: `https://docs.spring.io/spring-framework/reference/`

[5] "Spring Boot Reference Documentation," VMware. [Çevrimiçi]. Erişim: `https://docs.spring.io/spring-boot/docs/current/reference/html/`

[6] "Spring Data JPA — Reference Documentation," VMware. [Çevrimiçi]. Erişim: `https://docs.spring.io/spring-data/jpa/docs/current/reference/html/`

[7] Hibernate ORM Team, "Hibernate ORM 6.4 User Guide." [Çevrimiçi]. Erişim: `https://docs.jboss.org/hibernate/orm/6.4/userguide/html_single/`

[8] "PostgreSQL 15 Documentation," PostgreSQL Global Development Group. [Çevrimiçi]. Erişim: `https://www.postgresql.org/docs/15/`

[9] "React Documentation," Meta Platforms. [Çevrimiçi]. Erişim: `https://react.dev/`

[10] "Vite Guide," Evan You ve katkıda bulunanlar. [Çevrimiçi]. Erişim: `https://vitejs.dev/guide/`

[11] "Tailwind CSS Documentation," Tailwind Labs. [Çevrimiçi]. Erişim: `https://tailwindcss.com/docs`

[12] R. T. Fielding, "Architectural Styles and the Design of Network-based Software Architectures," Doktora tezi, Univ. of California, Irvine, 2000.

[13] M. Fowler, *Patterns of Enterprise Application Architecture*. Boston, MA, USA: Addison-Wesley, 2002.

[14] IEEE Computer Society, "IEEE 1016-2009 - IEEE Standard for Information Technology — Systems Design — Software Design Descriptions," 2009.

[15] IEEE Computer Society, "IEEE 830-1998 - IEEE Recommended Practice for Software Requirements Specifications," 1998.

[16] M. Fowler, *UML Distilled: A Brief Guide to the Standard Object Modeling Language*, 3rd ed. Boston, MA, USA: Addison-Wesley, 2003.

[17] J. Bloch, *Effective Java*, 3rd ed. Boston, MA, USA: Addison-Wesley, 2017.

[18] V. Driessen, "A successful Git branching model," *nvie.com*, 2010. [Çevrimiçi]. Erişim: `https://nvie.com/posts/a-successful-git-branching-model/`

[19] Necmettin Erbakan Üniversitesi, "BIL204 Yazılım Mühendisliği Ders Notları," Bilgisayar Mühendisliği Bölümü, 2025-2026 Bahar Dönemi.

Bu belge; Necmettin Erbakan Üniversitesi Bilgisayar Mühendisliği Bölümü BIL204 Yazılım Mühendisliği dersi kapsamında AOGS projesinin Proje Tasarım Raporu olarak teslim edilmek üzere hazırlanmıştır.

— BELGE SONU —